

面向 IaaS 云的高效概率工作流调度

Gabriele Russo Russo, Romolo Marotta, Flavio Cordari, Francesco Quaglia, Valeria Cardellini, Pierangelo Di Sanzo

摘要—云提供的计算资源既灵活又丰富，因此特别适合执行用户工作负载。然而，在具有截止期与费用约束的工作流调度中，IaaS 云环境带来了不容忽视的挑战。由于云资源的性能通常具有不确定性，假设执行时间确定的调度算法可能失效，因此需要概率方法。但现有概率算法的计算成本很高，主要原因是概率形式的工作流调度问题更加复杂，而且这些算法难以随问题实例规模增长而扩展。本文提出 EPOSS，一种基于概率形式化、面向 IaaS 云环境的新工作流调度算法。该方案将先进确定性执行时间调度算法的低执行延迟，与满足概率约束的能力结合起来。EPOSS 在设计时重点考虑计算效率，相比现有概率调度器，执行时间可降低一到两个数量级。此外，它在工作流规模和 IaaS 云所提供异构虚拟机类型数量增加时，也能保持良好扩展性。我们通过覆盖多种工作负载与 IaaS 云环境特征的实验比较，评估了该算法的收益。

关键词—工作流调度，IaaS 云，DAG 调度问题，调度算法，概率调度。

1 引言

基础设施即服务 (IaaS) 云资源供应模式，使用户能够快速获得定制的计算资源池；用户可以在按使用量计费的价格模式下，从多种资源类型中进行选择。然而，为给定用户负载确定最适合缩短执行时间或降低费用的资源池，并不简单。

这要求求解优化问题，而许多因素会进一步增加问题复杂度，例如资源池可以包含异构资源，以及 IaaS 云供应商市场提供的资源类型和价格十分多样。对于由多个任务构成的工作流，问题更具挑战：任务可以分布到多个计算资源上，但工作流执行计划不能违反任务依赖，例如某个任务需要接收另一个任务产生的输入数据。这类负载通常用有向无环图 (DAG) 建模 [1]，每个节点表示一个任务，从节点 a 到节点 b 的边，表示任务 a 与任务 b 之间的依赖。DAG 广泛用于各个应用领域，尤其用于描述许多科学工作负载 [2]。

除一些简单场景外，DAG 调度问题已知是 NP 完全的 [3]。因此，研究者提出多种多项式时间启发式，例如 [4], [5], [6], [7]。该问题也已在不同计算环境中研究，包括多处理器服务器，例如 [4], [6], [7]；网络环境，例如 [8], [9], [10]；以及更具挑战的 IaaS 云环境，例如 [11], [12], [13]。在最后一类环境中，需要寻找虚拟机 (VM) 资源池的配置，以及指定任务如何分配给 VM 的任务分配计划，使其满足：(a) 不违反任务依赖；(b) 在一组约束下优化一个或多个目标指标。优化指标与约束可随问题形式化方式变化。最常见的形式要求，在工作流执行时间的截止期约束下，最小化工作流执行费用 [13]。更一般地，可以将其形式化为带约束的多目标优化问题，需要寻找一个 Pareto 最优解集 [12]，其中包括这样的全部解：对任意其他候选解，它的费用或工作流执行时间至少有一项更高。

在 IaaS 环境中求解 DAG 调度问题的任何方法，都需要处理云资源的共享性与虚拟化特征。这些因素不可避免地使任务执行时间发生不可忽略的变化 [14], [15]，增加不确定性，

并影响实际费用 [14], [16]。例如，研究 [17] 和 [18] 在 AWS EC2 实例上运行科学工作流时，分别测得高达 24% 和 30% 的 CPU 性能变化。综述 [15] 提到，一些研究报告的运行时间变化测量值可达 90%，例如 [19]。

处理不确定性能需要采用概率问题形式化，以表达给定约束，例如截止期或最大费用，得到满足的期望概率；尤其需要允许执行时间服从任意分布。然而，如第 2 节所示，许多 DAG 调度方案都假设工作流任务具有固定的确定性执行时间。因此，即使性能变化很小，它们也可能失效。

文献中已有少量基于概率模型的方案 [12], [13]，但仍存在一些明显局限。首先，所提算法虽然使用启发式，计算成本仍然很高。此外，当 IaaS 提供商提供的 VM 类型数增加，或工作流规模增大时，其扩展性较差。第 5 节将结合实验数据讨论这些问题。最后，并非所有方案都考虑 IaaS 提供商施加的供应约束，例如一个用户能够同时使用的某类资源数量上限：AWS 的服务配额 [20]、Google Cloud 的分配配额 [21] 以及 Azure 的服务限制 [22]。若寻找最优调度方案时不考虑这些上限，执行工作流时某些 VM 可能不可用，进而导致截止期违约。

本文提出 **EPOSS**，名称来自 Efficient Probabilistic wOrk-flow Scheduling for IaaS cloudS，意为面向 IaaS 云的高效概率工作流调度。它是一种基于概率形式化的新调度算法，克服上述局限。首先，它显著降低寻找解的延迟。实验将表明，相比基于概率形式化的先进调度算法，其求解延迟可缩短 10 倍至超过 100 倍，同时找到同样好、甚至更好的解。此外，它具有更高扩展性，并覆盖更多约束，因而特别适合应对 IaaS 基础设施现有的资源供应模式。

本文希望设计一种算法，使其具有基于确定性执行时间假设的算法所提供的低延迟，同时能够求解更复杂的概率问题。因此，我们利用 MOHEFT [23] 的启发式效率；MOHEFT 是面向确定性执行时间的知名调度算法。将它与二分搜索以及 Monte Carlo 仿真结合，便可针对概率形式化问题识别最佳调度方案。

具体而言，本文作出以下贡献：

- EPOSS：面向 IaaS 云环境的高效 DAG 调度算法，针对概率问题形式化，在工作流执行时间截止期约束下最小化费用。

- G. Russo Russo, R. Marotta, F. Quaglia and V. Cardellini are with the Department of Civil and Information Engineering, University of Rome Tor Vergata, Italy. E-mail: russo.russo@ing.uniroma2.it, r.marotta@ing.uniroma2.it, Francesco.Quaglia@uniroma2.it, cardellini@ing.uniroma2.it.
- F. Cordari is with Sapienza University of Rome. E-mail: flavio.cordari@uniroma1.it.
- P. Di Sanzo is with Roma Tre University. E-mail: pierangelo.disanzo@uniroma3.it.

- P-EPOSS: EPOSS 的并行版本, 利用多个处理单元进一步缩短算法执行时间。
- M-EPOSS: 能够按 Pareto 最优性求解多目标优化问题的 EPOSS 版本, 同时最小化 workflow 执行时间与费用。
- 广泛的实验研究: 比较 EPOSS 与面向 IaaS 环境的先进算法, 考虑多种场景, 包括不同 workflow、VM 类型集合、任务执行时间分布及扩展性曲线, 以及用户可同时使用资源数量的不同限制。

本文其余部分安排如下: 第 2 节概述相关工作; 第 3 节介绍系统模型与问题形式化; 第 4 节介绍 EPOSS 及其并行版本、多目标版本; 最后, 第 5 节讨论实验结果。

2 相关工作

如前所述, DAG 调度问题已有多项基于启发式的多项式时间求解算法。这些算法针对不同计算环境设计, 包括单台多处理器服务器、计算机集群和网格。然而, IaaS 环境中的 DAG 调度问题与其他情形不同, 具有特定优化指标与约束 [11], [23]。例如, 由于云提供商采用按使用量付费模式, IaaS 云环境中通常最需要最小化的指标是费用; 另一个指标是 workflow 执行时间, 通常还要满足截止期。约束与假设可因具体情形而异, 例如 [11], [13], [27], [32]。由于 IaaS 云环境具有这些特殊性, 面向其他计算环境设计的 DAG 调度算法通常不适用: 它们优化的目标不同, 考虑的约束与假设也往往不同, 例如 [4], [5], [6], [8], [9]。

针对 IaaS 云环境的调度方案, 我们比较本文贡献与已有调度算法。比较结果汇总于表 1, 突出各算法的相关特征。所选算法来自多篇文献, 尤其参考了近期研究 [12]。表中显示, 已有方案考虑的优化指标与约束组合并不总是相同。虽然所有算法均将费用作为优化指标, 但只有其中一部分同时优化 workflow 执行时间。这些算法以基于 Pareto 最优性的多目标优化定义问题 [33]。在问题约束方面, 各算法对费用与 workflow 执行时间的处理也不同。多数方案对 workflow 执行时间施加约束, 即截止期; 部分方案还对费用施加最大预算约束。总体上, 只有少量算法, 即 Calzarossa 等 [12]、Verma & Kaushal [11] 以及本文算法, 同时优化执行时间和费用, 并允许对二者都设定约束。

算法之间的差异还涉及对 IaaS 云性能不确定性的假设。部分方案在算法设计和评估时均假设任务执行时间确定, 因此没有证据表明它们能在 IaaS 环境的任意性能条件下正确运行。

在考虑性能不确定性的方案中, 只有 Calzarossa 等 [12]、Zhou 等 [13] 以及本文算法, 允许任务执行时间服从任意分布, 并在该假设下评估算法。

其他算法采用更严格的假设。例如, Fard 等 [27] 假设任务执行时间未知, 但位于已知区间内, 并使用从该区间随机选择的执行时间评估算法。Meena 等 [31] 假设资源性能变化由已知百分比界限限制, 从而使任务执行时间变化有界。其实实验依据先前的经验观察 [17], 将百分比变化上界设为 24%, 并按均值 12%、标准差 10% 的正态分布生成数值。Rodriguez & Buyya [32] 更早采用了类似方法, 其实实验假设任务执行时间变化范围为 -10% 至 +10%, 并按正态分布生成。Anwar 等 [26] 提出运行时适应执行时间变化的方案: 当任务比预期更晚完成时, 重新调度后续任务, 尽量避免截止期违约。但该算法在构建初始计划时, 不考虑任务执行时间的不可预测性。

关于用户可同时使用资源数量的限制, Zhu 等 [25] 考虑 VM 总数上限, 并认为该方法可以扩展至其他资源限制。本文算法同时考虑 VM 或 vCPU 的最大总数, 以及每种 VM 类型的最大数量。其他方案中, 只有 Zhou 等 [13] 考虑资源限制的存在; 但他们仅在实验中考虑这些限制, 假设在执行工

作流期间, 用户已达到允许的 VM 数量上限, 而调度器仍可能需要新的 VM。他们指出, 此时必须等待某个 VM 被释放, 因为算法本身无法处理这种情况。

最后, 在问题形式化方面, 表 1 中考虑任务执行时间任意分布的三个算法, 包括本文算法, 也是仅有的采用概率形式化的方案。

与确定性形式化不同, 概率形式化允许指定希望 workflow 满足执行截止期、或不超过给定费用的概率。因此, 可以更可靠地描述和处理 IaaS 云环境中的性能不确定性。

总体上, 表 1 表明, 本文方案涵盖其他现有算法的全部指标与要求, 并进一步纳入其他方案未完整覆盖的约束。

在求解方法方面, 本文算法与其他方案明显不同。Meena 等 [31] 的算法基于遗传方法。Rodriguez & Buyya [32] 采用粒子群优化, 即受鸟群社会行为启发的进化计算技术。Abrishami 等 [24] 提出两阶段算法: 先根据整个 workflow 截止期确定各任务截止期, 再依据分配的截止期调度每个任务。Zhu 等 [25] 的方案部分受到分布式系统多资源任务装箱的启发, 即依据任务的多种资源需求, 将多个任务装入不同服务器。

本文算法则利用 MOHEFT [23] 的列表启发式保持高效率, 第 4 节将介绍该算法; 同时结合基于二分搜索的技术与 Monte Carlo 评估, 处理概率形式化问题。因此, 本文方案与另外两种概率算法明显不同。Calzarossa 等 [12] 采用遗传方法, 与本文算法唯一相似之处是使用 Monte Carlo 仿真。但实验将表明, 本文方案需要的 Monte Carlo 评估次数少得多。这使算法能够大幅缩短执行时间; 当解空间扩大时, 执行时间通常是调度算法可用性的主要障碍。另一种概率算法, 即 Zhou 等 [13] 的方案, 依赖基于 A* 的搜索来确定执行 workflow 任务的 VM 类型。使用该搜索方法, 是因为剪枝有望缩小庞大的搜索空间。然而, 本文实验将揭示这种方法的重要局限, 并展示本文算法在执行时间和扩展性方面的明显优势。

3 系统模型与问题形式化

考虑由计算任务集合 V 构成的 workflow。任务可按任意顺序执行, 除非任务 $v_j \in V$ 接收另一个任务 $v_i \in V$ 产生的输入数据。此时两任务存在依赖, v_j 只能在 v_i 执行完成之后开始。整个 workflow 以有向图描述, 节点表示任务, 有向边表示依赖。假设任务之间没有环和条件依赖, 则表示 workflow 的图是有向无环图 (DAG), 记为 $G = (V, E)$; 其中, V 是节点或任务集合, E 是边或依赖集合。因此, 一条边是有序对 $(v_i, v_j) \in V \times V$, 表示 v_j 只能在 v_i 完成后开始。注意, 给定节点集合 $\{u, v_1, v_2, \dots, v_n\} \subseteq V$, 若存在边集合 $\{(v_1, u), (v_2, u), \dots, (v_n, u)\} \subseteq E$, 则任务 u 只能在全部任务 $v_1, v_2, \dots, v_n$ 执行结束后开始。

假设执行 workflow 的 VM 池可以异构, 即由不同类型的 VM 构成。VM 类型表示其虚拟计算资源配置, 包括 CPU 的类型和数量、内存容量等。用 Θ 表示可用 VM 类型集合。

用 M^R 表示可同时分配给一个用户的最大资源量。实际中, M^R 可以表示 VM 数或虚拟 CPU (vCPU) 数, 这两种参数通常被 IaaS 提供商用于限制用户可同时使用的资源总量。此外, IaaS 提供商常对用户可同时使用的每种类型 VM 数量设置专门限制, 因此以 $M_j^{type}, \forall j \in \Theta$, 表示允许使用的 j 类型 VM 最大数量。

调度问题的解是一个任务分配计划, 指定资源池中的 VM 类型, 以及每个 VM 上需要执行的任务序列。形式上, 解 s 是由 n 个有序对 (j_i, L_i) 构成的列表, 其中 $1 \leq i \leq n$, 且:

- $j_i \in \Theta$ 是池中第 i 个 VM 的类型;
- L_i 是池中第 i 个 VM 上待执行的任务列表。

注意, $L_1 \cup L_2 \cup \dots \cup L_n = V$ 且 $L_1 \cap L_2 \cap \dots \cap L_n = \emptyset$ 。解 s 的费用记为 $C(s)$, 取决于池中每个 VM 的类型与使用时间。假设 c_j^{type} 是 j 类型 VM 的单位时间使用费用, 则

表 1: 面向 IaaS 云环境的 DAG 调度算法的相关特征。勾号表示算法设计完整考虑该特征, “部分考虑”表示仅部分考虑, “可扩展”表示算法在设计时允许较容易地扩展, 以覆盖该特征。

	优化指标		约束		不确定性		资源限制		形式化
	workflow 执行时间	费用	workflow 执行时间	费用	云资源性能	任意任务执行时间分布	VM 或 vCPU 总数	每种类型的 VM 数量	概率形式
Abrishami 等 [24]		✓	✓						
Zhu 等 [25]		✓	✓				✓	可扩展	
Anwar 等 [26]		✓	✓						
Fard 等 [27]	✓	✓			部分考虑				
Hu 等 [28]	✓	✓							
Li 等 [29]		✓	✓		✓				
Liu 等 [30]		✓	✓						
Meena 等 [31]		✓	✓		部分考虑				
Rodriguez & Buyya [32]		✓	✓		部分考虑				
Verma & Kaushal [11]	✓	✓	✓	✓					
Calzarossa 等 [12]	✓	✓	✓	✓	✓	✓			✓
Zhou 等 [13]		✓	✓		✓	✓	部分考虑		✓
EPOSS	✓	✓	✓	✓	✓	✓	✓	✓	✓

$C(s)$ 等于池中所有 VM 的“使用时间乘以 c_j^{type} ”之和。VM 的期望使用时间由其上所有任务的执行时间之和给出。注意, 不同 VM 类型的计算资源配置不同, 因此同一任务在不同 VM 类型上执行, 所需时间可能不同。此外, 由于云资源的性能通常不确定, 即使固定 VM 类型, 任务执行时间也会变化。文献采用多种任务执行模型计算执行时间。例如, 在 [12], [13] 中, 它是任务 CPU 时间、I/O 时间和网络通信时间之和; 更详细的模型, 例如 [34], 还包括排队时间等因素。不同模型是否适合取决于具体需求, 因此本文抽象掉影响任务执行时间的具体因素, 不失一般性地将任务 $v \in V$ 在 $j \in \Theta$ 类型 VM 上的执行时间, 建模为服从任意分布的随机变量, 记为 $t_{v,j}$ 。这意味着池中各 VM 的使用时间以及费用 $C(s)$, 也都是依赖任务执行时间的随机变量。

给定解 s , 用 $T(s)$ 表示对应的工作流执行时间, 即任务按 s 的分配计划执行时 G 的完工时间。因此, $T(s)$ 也是随机变量, 对应 G 关键路径上任务执行时间之和。

为简化介绍, 先围绕最常见的问题形式介绍调度算法: 在执行时间截止期 d 约束下最小化费用, 称为单目标形式化。随后说明如何修改算法, 以处理需要寻找 Pareto 最优解集的带约束多目标优化形式。

基于上述模型, 将单目标问题形式化如下:

$$\min_{s \in \Omega} \mathbb{E}[C(s)] \quad (1)$$

$$\text{约束条件为} \quad P(T(s) \leq d) \geq p_T \quad (2)$$

$$N^R(s) \leq M^R \quad (3)$$

$$N_i^{type}(s) \leq M_i^{type}, \forall i \in \Theta, \quad (4)$$

其中:

- s 是问题的一个解, Ω 是解空间, 即任务执行不违反 G 中任务依赖的全部解组成的集合;
- $\mathbb{E}[C(s)]$ 是 s 的期望费用;
- 式 (2) 确保满足截止期 d 的概率至少为 p_T ;
- 式 (3) 保证 s 同时使用的 VM 或 vCPU 最大数量 $N^R(s)$ 不超过 M^R ;
- 式 (4) 限制 s 同时使用的每种类型 $i \in \Theta$ 的 VM 最大数量 $N_i^{type}(s)$, 使其不超过 M_i^{type} 。

4 调度算法

如第 1 节所述, EPOSS 利用面向确定性执行时间的先进算法 MOHEFT [23] [35] 的启发式。我们先介绍 MOHEFT, 再介绍 EPOSS, 并讨论 MOHEFT 在其中的作用。

4.1 MOHEFT

MOHEFT 是多目标异构最早完成时间的缩写 [23], [35], 是一种在异构机器上调度工作流的多项式时间启发式算法。它扩展了知名的 HEFT 算法 [36]; HEFT 是基于列表的启发式, 用于优化工作流应用的完工时间。相比 HEFT, MOHEFT 通过确定 Pareto 前沿来优化多个目标, 通常包括完工时间与费用。

MOHEFT 的伪代码见算法 1。输入包括 DAG G 、每种 VM 类型上的确定性任务执行时间集合 D , 即 $\{T_{v,j}\}, \forall (v,j) \in V \times \Theta$, 以及需要沿 Pareto 前沿计算的解数 K 。为确定该前沿, MOHEFT 按如下方式工作: 维护集合 S , 最多包含 K 个部分调度解, 其中既包括对各目标, 即完工时间与费用分别最优的解, 也包括二者之间的其他权衡解。 S 初始为 K 个空解, 见行 1。然后, 根据 B-Rank 指标对工作流所有任务排序, 参见 [36]; 该指标依据任务距工作流结束的距离, 即工作流预期结束时间与任务预期开始时间之差, 形成拓扑排序。基本上, 必须先执行的任务具有更高排序值, 见行 2。MOHEFT 从排序值最高的任务开始, 每次调度一个任务。逐任务迭代时, 行 3 的每次迭代都创建一个新的空候选解集 S' , 见行 4。通过为 S 中的解分配排序中第 r 个任务来扩展解, 从而填充 S' 。具体而言, 对于每个解 s_k , 考虑 VM 集合 I , 包含 s_k 中已使用的全部 VM, 以及每种类型的一个新实例, 见行 6。将第 r 个任务追加到 I 中每个 VM 的任务列表末尾, 生成新的部分解, 见行 7-8。每个新解都加入 S' , 见行 9。开始下一任务的循环前, 依据拥挤距离 [37], 从 S' 中选择最多 K 个解构建新的 S , 见行 10。拥挤距离衡量一个解与邻近解之间的接近程度, 也就是解空间中某区域的“拥挤”程度。因此, 选择距离最高的 K 个解。MOHEFT 的复杂度为 $O(|V| \cdot |\Theta| \cdot K)$ 。

4.2 EPOSS 设计

MOHEFT 假设任务执行时间确定, 但这一假设在实际 IaaS 环境中很难成立。时间存在变化时, 无法预先知道每个任务准确需要的时长。为绕开这一问题, 可以尝试近似值, 例如任务执行时间的均值或中位数。但依据这类数据找到的任何解, 都无法确保截止期永不违反。更一般地, 基于确定性执行时间假设的调度方法, 不能控制多次工作流执行中预期截止期违约次数的上限。

按照单目标问题形式, 本文算法旨在确保满足截止期 d 的概率至少为 p_T 。基本思路是, 利用任务执行时间的不同统计量, 即不同分位数, 作为 MOHEFT 输入, 寻找候选解集; 再识别其中成本最低、且将预期截止期违约次数限制到满足概率 p_T 要求的解。具体而言, EPOSS 通过二分搜索探索不同

算法 1: MOHEFT: 多目标 workflow 调度

```

输入:  $G = (V, E)$  ▷ DAG
输入:  $D = \{T_{v,j}, \forall (v,j) \in V \times \Theta\}$  ▷ 每种 VM 类型上的任务执行时间
输入:  $K \in \mathbb{N}^+$  ▷ 待计算的解数
输出:  $S = \{s_1, \dots, s_K\}$  ▷ 包含  $K$  个解的 Pareto 前沿
1  $S \leftarrow \{s_1, \dots, s_K\}$ , 其中  $s_i = \emptyset$  ▷ 创建  $K$  个空解
2  $Rnk \leftarrow \text{B-RANK}(G)$ 
3 对  $u \leftarrow 1, |V|$  执行
4    $S' \leftarrow \emptyset$ 
5   对  $s_k \in S$  执行
6      $I \leftarrow s_k \cup \{VM_j\}, \forall j \in \Theta$  ▷ 考虑  $s_k$  中已有 VM, 以及每种类型的一个新 VM
7     对  $VM_i \in I$  执行
8        $s \leftarrow \text{ADDTASK}(s_k, Rnk_u, VM_i)$  ▷ 将任务  $Rnk_r$  分配给  $s_k$  中的  $VM_i$ 
9        $S' \leftarrow S' \cup \{s\}$ 
10  $S \leftarrow \text{FIRSTCROWDINGDISTANCE}(S', K, D)$ 

```

算法 2: 对 MOHEFT 的修改

```

输入:  $d \in \mathbb{R}^+$  ▷ 截止期
输入:  $M^R \in \mathbb{R}^+$  ▷ VM 或 vCPU 最大数量
输入:  $\{M_j^{type}, \forall j \in \Theta\}$  ▷ 每种类型的 VM 最大数量
输入:  $G = (V, E)$  ▷ DAG
输入:  $D = \{T_{v,j}, \forall (v,j) \in V \times \Theta\}$  ▷ 每种 VM 类型上的任务执行时间
输入:  $K \in \mathbb{N}^+$  ▷ 待计算的解数
输出:  $S = \{s_1, \dots, s_K\}$  ▷ 包含  $K$  个解的 Pareto 前沿
1  $S \leftarrow \{s_1, \dots, s_K\}$ , 其中  $s_i = \emptyset$  ▷ 创建  $K$  个空解
2  $Rnk \leftarrow \text{B-RANK}(G)$ 
3 对  $u \leftarrow 1, |V|$  执行
4    $S' \leftarrow \emptyset$ 
5   对  $s_k \in S$  执行
6      $I \leftarrow s_k \cup \{VM_j\}, \forall j \in \Theta$  ▷ 考虑  $s_k$  中已有 VM, 以及每种类型的一个新 VM
7     对  $VM_i \in I$  执行
8        $s \leftarrow \text{ADDTASK}(s_k, Rnk_u, VM_i)$  ▷ 将任务  $Rnk_r$  分配给  $s_k$  中的  $VM_i$ 
9       如果  $\text{FEASIBLE}(s, d, M^R, \{M_j^{type}, \forall j \in \Theta\})$  则
10          $S' \leftarrow S' \cup \{s\}$ 
11  $S \leftarrow \text{FIRSTCROWDINGDISTANCE}(S', K, D)$ 

```

解。每一步搜索, 计算全部任务执行时间的 α 阶分位数, 即 α 分位数, 并以这些数值为输入运行 MOHEFT。EPOSS 从返回解中选择费用最低者, 再通过 Monte Carlo 仿真评估它。仿真从任务的预期分布随机抽取执行时间, 按照所选解确定的 VM 配置与任务分配计划, 模拟多次 workflow 执行, 返回测得的截止期满足经验概率以及平均费用。根据这些结果, EPOSS 决定继续在相对于 α 的分位数搜索区间上半部还是下半部搜索。其理由是: 更高的分位数阶数可能产生更保守的解, 即满足截止期的概率更高, 但费用也更高; 更低的阶数可能得到更便宜的解, 但截止期违约风险更高。当分位数搜索区间小于某个大小时, 停止搜索并选择已找到的解。以下详细介绍 EPOSS 的实现与工作过程。

4.2.1 对 MOHEFT 的修改

首先, 修改算法 1 中的 MOHEFT 实现: 只有 workflow 执行时间不违反截止期 d , 且解 s 所需资源量不超过对应上限时, 才将新生成的解 s 加入 S' 。换言之, 仅当新解不违反第 (2) 式、第 (3) 式和第 (4) 式所表达的约束时, 才将其加入 S' ; 这些约束属于第 3 节的单目标形式化。

修改后的 MOHEFT 伪代码见算法 2。在初始部分加入截止期及上述两个约束的输入数据, 并加入一个 *if* 条件, 见行 9, 调用函数 $\text{FEASIBLE}(s, d, M^R, M_i)$ 。该函数检查是否满足第 (2) 式、第 (3) 式和第 (4) 式表达的约束。若满足, 则将 s 加入 S' , 见行 10。

函数 Feasible 检查解 s 是否满足所考虑约束, 伪代码见算法 3。它首先检查预期 workflow 完工时间是否位于截止期内,

见行 2-4。为此, 取全部已调度任务的预期完成时间最大值, 作为完工时间。任务预期完成时间按 [36] 中的方法递归计算, 同时计入任务读写中间数据所需时间。这里假设任务通过共享存储写出输出, 例如分布式文件系统或 AWS S3 一类对象存储服务, 之后由后继任务读取。为估计通信开销, 将每个任务的预期输出数据量除以所用机器的估计网络带宽。如果任务及其全部后继都位于同一个 VM, 则不产生这一开销, 因为可用 VM 本地存储保存中间数据。若满足截止期, 则继续检查工作流执行所产生的预期资源使用量, 即所分配的 vCPU 与 VM 实例, 是否超过用户允许的最大资源量。具体而言, 首先按照 s 建立按时间排序的 VM 分配与释放列表, 见行 6-12; 再验证任意时刻都不违反资源约束, 见行 13-17。

4.2.2 EPOSS

EPOSS 的伪代码见算法 4。算法记录当前识别到的最佳解及其成本, 初始为空解和无穷大成本, 见行 1-2。分位数搜索区间由其最低值与最高值定义, 初始分别为 $\alpha_l = 0$ 和 $\alpha_h = 1$, 见行 3。之后开始搜索, 见行 4; 只要区间宽度大于阈值 ϵ 就继续, 该阈值位于 $[0, 0.5]$ 内, 见行 16。每一步先计算区间中点 α , 见行 5, 再计算每个任务执行时间的 α 分位数, 见行 6, 并用这些数据作为修改版 MOHEFT 的输入, 见行 7。从返回的 K 个解中, 选择成本最低的 $\bar{s}$, 见行 8, 调用函数 Evaluate , 见行 9。它运行 Monte Carlo 仿真评估解 $\bar{s}$, 返回对应的截止期满足概率 $\bar{p}_T$ 与平均费用 $\bar{C}$ 。接着, 检查 $\bar{p}_T$ 是否大于等于 p_T , 见行 10; 若是, 则将搜索区间缩到下半部 $[\alpha_l, \alpha]$, 见行 11, 并在 $\bar{s}$ 的估计费用低于此前最佳解时, 将 $\bar{s}$ 提升为最佳

算法 3: Feasible 函数的实现

```

1  算法 2 的全部输入数据, 再加上 输入:  $s$  ▷ 待评估的解
   输出: 给定解是否可行
2  函数 FEASIBLE( $s, d, M^R, M^{type}$ ):
   ▷ 函数  $EST$  与  $EFT$  分别返回预期开始时间与预期完成时间
3    $makespan \leftarrow \max_{v \in V} EFT(v; s)$ 
   ▷ 检查截止期约束
4   如果  $makespan > d$  则
5   |   返回 False
   ▷ 检查资源约束
6    $Allocations \leftarrow \emptyset$ 
7   对  $VM \in s$  执行
8   |    $t_0 \leftarrow \min_{v \in L_{VM}} EST(v; s)$  ▷ VM 启动时间
9   |    $t_1 \leftarrow \max_{v \in L_{VM}} EFT(v; s)$  ▷ VM 关闭时间
10  |    $Allocations \leftarrow Allocations \cup \{(t_0, \theta_{VM}, 1), (t_1, \theta_{VM}, -1)\}$ 
11   $r \leftarrow 0$  ▷ 已分配资源
12   $r_j \leftarrow 0$  ▷ 每种类型  $j \in \Theta$  的已分配实例
13   $A \leftarrow \text{sort } Allocations \text{ 按时间递增排序}$ 
14  对  $(t, j, x) \in A$  执行
15  |    $r \leftarrow r + xCPU_j$ 
16  |    $r_j \leftarrow r_j + x$ 
17  |   如果  $r > M^R$  或  $r_j > M_j^{type}$  则
18  |   |   返回 False

```

算法 4: EPOSS

```

输入:  $G = (V, E)$  ▷ 工作流
输入:  $\{t_{v,j}, \forall (v, j) \in V \times \Theta\}$ , 其中  $t_{v,j}$  是随机变量 ▷ 每种 VM 类型上的任务执行时间
输入:  $p_T \in [0, 1]$  ▷ 要求的截止期满足概率
输入:  $\epsilon > 0$  ▷ 搜索停止阈值
输出:  $s_{best}$  ▷ 已找到的最佳调度解
1   $s_{best} \leftarrow \emptyset$ 
2   $C_{best} \leftarrow \infty$  ▷ 最佳调度解的成本
3   $\alpha_l \leftarrow 0, \alpha_h \leftarrow 1$  ▷ 初始分位数区间
4  执行
5  |    $\alpha \leftarrow \frac{1}{2}(\alpha_l + \alpha_h)$ 
6  |    $T_{v,j}^\alpha \leftarrow t_{v,j}, \forall (v, j) \in V \times \Theta$  的  $\alpha$  分位数 ▷ 计算任务执行时间的  $\alpha$  分位数
7  |    $\bar{S} \leftarrow \text{MOHEFT}(G, \{T_{v,j}^\alpha, \forall (v, j) \in V \times \Theta\}, K)$ 
8  |    $\bar{s} \leftarrow \text{argmin}_{s \in \bar{S}} C(s)$  ▷ MOHEFT 中成本最低的解
9  |    $\bar{p}_T, \bar{C} \leftarrow \text{EVALUATE}(G, \bar{s})$ 
10 |   如果  $\bar{p}_T \geq p_T$  则
11 |   |    $\alpha_h \leftarrow \alpha$ 
12 |   |   如果  $\bar{C} < C_{best}$  则
13 |   |   |    $s_{best} \leftarrow \bar{s}$ 
14 |   |   |    $C_{best} \leftarrow \bar{C}$ 
15 |   否则  $\alpha_l \leftarrow \alpha$ 
16 当  $\alpha_h - \alpha_l > \epsilon$ 

```

解, 见行 12–14; 否则, 将搜索区间缩到上半部 $[\alpha, \alpha_h]$, 见行 15。

EPOSS 的搜索步数取决于所选 ϵ 值, 见行 16。具体而言, 执行 $\log_2 \frac{1}{\epsilon}$ 步。实验观察到, ϵ 取 0.02 时, EPOSS 已达到与所比较另外两种先进概率算法相当的准确度。采用该配置, EPOSS 执行 6 步搜索, 因此最多运行 MOHEFT 6 次。由于 MOHEFT 运行次数为常数, EPOSS 与 MOHEFT 具有相同复杂度阶。

4.3 并行 EPOSS

上述 EPOSS 实现依次执行搜索步骤。为利用硬件并行性, 我们设计了并行版本 **P-EPOSS**。它不采用顺序搜索, 而是假设具有 $P \geq 2$ 个处理单元, 将完整分位数阶数区间 $[0, 1]$ 划分为 P 个等宽子区间, 边界点序列为 $\alpha_0, \alpha_1, \alpha_2, \dots, \alpha_{P-1}, \alpha_P$, 其中 $\alpha_0 = 0$ 且 $\alpha_P = 1$ 。每个子区间 $[\alpha_{i-1}, \alpha_i]$, $i \in [1, P]$, 分配给 P 个并行线程中的一个, 执行算法 4 行 5–9 的同一组

操作。具体而言, 若线程负责区间 $[\alpha_{i-1}, \alpha_i]$, 则计算任务执行时间的 $\frac{1}{2}(\alpha_{i-1} + \alpha_i)$ 分位数, 并用它作为修改版 MOHEFT 的输入。然后选取返回解中成本最低者, 用 Monte Carlo 仿真评估其截止期满足概率与平均费用。之后, 在 P 个并行处理单元所选全部解中, P-EPOSS 选择费用最低者, 再将该解所属子区间划分为 P 个等宽子区间, 交由 P 个并行线程重新执行。只要子区间宽度仍大于阈值 ϵ , 便继续划分循环。循环结束后, 返回最后选中的解。

注意, P-EPOSS 每轮都将区间划分为 P 个子区间, 并对每个子区间选择中点分位数, 因此经过 c 轮后, 子区间宽度为 $1/(2 \cdot P^c)$ 。所以它在 $\log_{2P} \frac{1}{2\epsilon}$ 步后终止。每步中, P 个并行线程各自执行的操作, 与 EPOSS 一步搜索相同, 因此可以假设 EPOSS 的一步搜索, 与 P-EPOSS 的一步划分耗时相近。但达到与 EPOSS 同样精度所需的搜索步数, 会减少如下

倍数:

$$\frac{\log_2 \frac{1}{\epsilon}}{\log_P \frac{1}{2\epsilon}}, \quad (5)$$

该比值对应 P-EPOSS 的加速比。例如, ϵ 取前述 0.02 时, EPOSS 需要 6 步搜索; P-EPOSS 使用 4 个并行线程需要 3 步, 使用 8 个并行线程需要 2 步。第 5.2 节给出详细测量, 展示 P-EPOSS 在算法执行时间方面的收益。

4.4 带约束多目标优化问题

引入一些额外变量, 即允许的最大费用 c , 以及费用小于 c 的概率 p_C , 便可将带约束多目标优化问题形式化如下:

$$\min_{s \in \Omega} (\mathbb{E}[C(s)], \mathbb{E}[T(s)]) \quad (6)$$

$$\text{约束条件为} \quad P(T(s) \leq d) \geq p_T \quad (7)$$

$$P(C(s) \leq c) \geq p_C \quad (8)$$

$$N^R(s) \leq M^R \quad (9)$$

$$N_i^{type}(s) \leq M_i^{type}, \forall i \in \Theta. \quad (10)$$

在这一形式下, 问题目标是寻找 Pareto 最优解集。更形式化地, 该集合 S_P 满足: 对于每个 $s \in S_P$, 第 (7) 式与第 (8) 式的约束均成立, 并且不存在其他 $s' \in S_P$ 使 $\mathbb{E}[T(s)] > \mathbb{E}[T(s')]$ 和 $\mathbb{E}[C(s)] > \mathbb{E}[C(s')]$ 同时成立。

为求解多目标形式的问题, 对 EPOSS 作少量修改, 得到算法 5, 称为 **M-EPOSS**。M-EPOSS 在 $[0, 1]$ 区间构建 ϵ 分位数集合 A , 见行 1。随后, 对每个 $\alpha \in A$, 见行 2, 计算任务执行时间的 α 分位数, 见行 3, 并用这些数值作为修改版 MOHEFT 的输入, 见行 4。对每个 α 返回的解, 使用 Monte Carlo 仿真评估, 见行 5-6; 这时返回满足截止期 d 的概率 $\bar{p}_T$ 、费用小于 c 的概率 $\bar{p}_C$, 以及期望完工时间和费用, 见行 6。满足第 (7) 式与第 (8) 式约束的解, 被加入 Pareto 前沿候选解集 F , 见行 7-8。最后, 依据期望完工时间与费用, 返回 F 中全部非支配解。M-EPOSS 很容易并行化: 让 P 个并行线程各自对不同的 $\alpha \in A$ 执行行 3-8 即可。

算法 5: 用于带约束多目标优化问题的 EPOSS

输入: $G = (V, E)$ ▷ 工作流
输入: $\{t_{v,j}, \forall (v,j) \in V \times \Theta\}$, 其中 $t_{v,j}$ 是随机变量 ▷ 每种 VM 类型上的任务执行时间
输入: $p_T \in [0, 1]$ ▷ 满足截止期的概率
输入: $p_C \in [0, 1]$ ▷ 费用小于 c 的概率
输入: $\epsilon > 0$ ▷ 搜索停止阈值
输出: F ▷ Pareto 前沿

- 1 $A \leftarrow \{\alpha_k : k \in \mathbb{N}^+, \alpha_k = k \cdot \epsilon, \alpha_k < 1\}$ ▷ 区间 $[0, 1]$ 中间隔为 ϵ 的分位数集合
- 2 **对** $\alpha \in A$ **执行**
- 3 $T_{v,j}^\alpha \leftarrow t_{v,j}, \forall (v,j) \in V \times \Theta$ 的 α 分位数 ▷ 计算每个任务执行时间的 α 分位数
- 4 $\bar{S} \leftarrow \text{MOHEFT}(G, \{T_{v,j}^\alpha, \forall (v,j) \in V \times \Theta\}, K)$
- 5 **对** $\bar{s} \in \bar{S}$ **执行**
- 6 $\bar{p}_T, \bar{p}_C, \bar{T}, \bar{C} \leftarrow \text{EVALUATE}(G, \bar{s})$
- 7 **如果** $\bar{p}_T \geq p_T$ **且** $\bar{p}_C \geq p_C$ **则**
- 8 $F \leftarrow F \cup (\bar{s}, \bar{T}, \bar{C})$ ▷ 加入解的数据
- 9 从 F 移除被支配的解

5 实验评估

我们通过广泛的实验研究评估调度方法, 并与其他方案比较。实验考虑多种工作流配置、云环境设置、约束设置和执行时间分布。相比以往概率工作流调度算法的实验 [12], [13], 本文使用更大的工作流集合与 VM 类型集合。本节介绍实验细节, 并讨论所得结果。

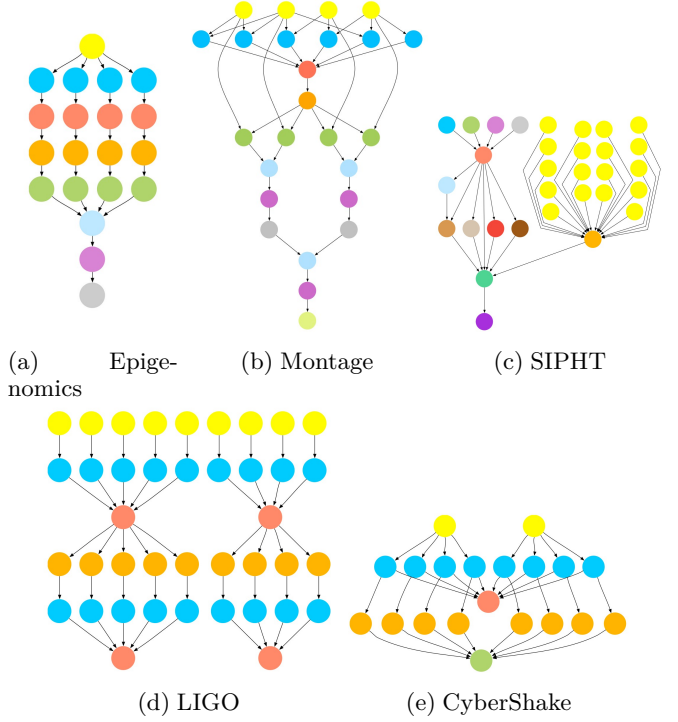


图 1: 实验所用工作流的图示, 图片取自 Pegasus 工作流仓库: <https://pegasus.isi.edu>。

5.1 实验设置

本小节详细介绍实验设置。

5.1.1 工作流

所选工作流包括先前调度算法研究 [2] 最常用的五种: *Epigenomics*、*SIPHT*、*CyberShake*、*Montage* 和 *LIGO*。它们来自不同应用领域, 一起覆盖科学工作流常见的多种拓扑结构。Epigenomics 用于自动化基因组序列处理操作; SIPHT 用于自动搜索细菌调节 RNA; CyberShake 用于刻画地震危险; Montage 是将多幅输入图像拼接成天空图像的天文学应用; LIGO 则用于分析引力波。图 1 给出每种工作流的具体参考结构。更详细的工作流特征可参阅 [1]。

5.1.2 任务执行时间

如前所述, 将任务执行时间建模为随机变量。因此, 给定工作流任务 $v \in V$ 及 VM 类型 $j \in \Theta$, 假设平均任务执行时间 $\bar{t}_{v,j}$ 以及执行时间概率分布已知。还依据已有研究的结果, 例如 [12], [38], [39], 为实验选择以下概率分布:

- *Gamma* 分布, 形状参数为 1, 尺度参数为 $\bar{t}_{v,j}$;
- 半正态分布, 均值为 $\bar{t}_{v,j}$;
- 均匀分布, 支持区间为 $[0, 2 \cdot \bar{t}_{v,j}]$;
- 确定性时长, 即始终等于 $\bar{t}_{v,j}$ 。

对于平均任务执行时间与估计输出数据量的具体数值, 我们采用 [1] 中的数据, 该研究详细描述了实验所用五种工作流。所用数据来自其表 3、4 和 6-8 的 *Runtime* (运行时间) 与 *I/O write* (I/O 写入) 列。为简洁起见, 这里不列出全部数据, 而用表 2 报告每种工作流平均任务执行时间与估计输出数据量的最大值、最小值, 展示其范围。注意, 如第 4 节所述, 输出数据从源 VM, 即产生数据的任务所在 VM, 传输到目标 VM, 即接收数据的任务所在 VM, 平均所需时间计算为待传输数据总量除以源、目标 VM 的最小网络带宽。

表 2: 文献 [1] 所报告平均任务执行时间和任务输出数据量的最小值与最大值

工作流	平均任务 执行时间 (秒)		输出数据量 (MB)	
	最小值	最大值	最小值	最大值
Epigenomics	0.48	201.89	0.90	242.29
Montage	0.64	384.49	0.10	775.45
SIPHT	0.03	3311.12	0.03	567.01
LIGO	0.13	0.14	0.01	0.13
CyberShake	0.55	265.73	0.02	176.48

表 3: EC2 VM 类型参考集合

参考 EC2 家族	参考 EC2 类型	vCPU 数	带宽 (Mbps)	价格 (美元/小时)
c4	c4.large	2	62.50	0.114
c4	c4.xlarge	4	125.00	0.227
c4	c4.2xlarge	8	125.00	0.454
c4	c4.4xlarge	16	125.00	0.909
c4	c4.8xlarge	36	1250.00	1.817
c5	c5.large	2	1250.00	0.097
c5	c5.xlarge	4	1250.00	0.194
c5	c5.2xlarge	8	1250.00	0.388
c5	c5.4xlarge	16	1250.00	0.776
c5	c5.9xlarge	36	1250.00	1.746
c5	c5.12xlarge	48	1500.00	2.328
c5	c5.18xlarge	72	3125.00	3.492
c5	c5.24xlarge	96	3125.00	4.656
m5	m5.large	2	1250.00	0.115
m5	m5.xlarge	4	1250.00	0.230
m5	m5.2xlarge	8	1250.00	0.460
m5	m5.4xlarge	16	1250.00	0.920
m5	m5.8xlarge	32	1250.00	1.840
m5	m5.12xlarge	48	1250.00	2.760
m5	m5.16xlarge	64	2500.00	3.680
m5	m5.24xlarge	96	3125.00	5.520

5.1.3 VM 类型

我们根据 IaaS 提供商所提供 VM 类型的特征定义实验中的类型集合, 具体以 AWS EC2 的 VM 类型为参考¹。更具体地, 考虑 EC2 的 c4 家族 (5 种实例)、c5 家族 (8 种实例) 和 m5 家族 (8 种实例)。与其他 IaaS 提供商类似, AWS EC2 同一家族的全部 VM 类型使用相同种类的硬件, 例如相同 CPU 型号, 但资源规模不同; 尤其是同一家族中的每种 VM 类型, 其 vCPU 数量都与其他类型不同。基于此, 定义 21 种不同 VM 类型, 每一种的相关特征, 即核数、带宽和小时价格, 均取自上述某种 EC2 类型。实验考虑的 21 种 VM 数据列于表 3。该集合覆盖 2 至 96 个 vCPU, 价格范围为每小时 0.114 至 5.520 美元。

关于任务在不同 VM 类型上的平均执行时间, 采用 [1] 报告的数值作为单 vCPU 的基准, 参见第 5.1.2 节的讨论; 再应用通用扩展性定律 (USL), 估计 vCPU 更多时如何变化。USL 将并行度为 N 的机器, 在本文中对应具有 N 个 vCPU 的 VM, 其相对计算能力或加速比估计为:

$$C(N) = \frac{N}{1 + \alpha(N-1) + \beta N(N-1)}, \quad (11)$$

其中, 系数 α 与 β 分别取决于访问机器共享资源的排队延迟, 以及确保并发共享数据访问一致性所需的延迟。 α 与 β 的不同组合对应不同扩展性场景。对于本文五种科学负载, 我们观察到 $\alpha = 0.01$ 、 $\beta = 0$ 的设置, 记为扩展性场景 A, 能够较好地拟合其扩展曲线形状。这符合科学工作流的预期: 对同一数据的并发读写操作通常有限, 因此数据一致性延迟很不显著。但本文也考虑其他可能的扩展性曲线, 并纳入另外两种场景。具体而言, 考虑两个相反情形: $\alpha = 0.01$ 、 $\beta = 0$, 即场景 B; 以及 $\alpha = 0.0001$ 、 $\beta = 0.001$, 即场景 C。三种场景的典型曲线形状见图 2。

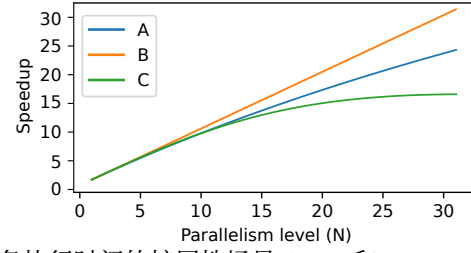


图 2: 任务执行时间的扩展性场景 A、B 和 C。

图中术语: Parallelism level (N) 为并行度; Speedup 为加速比。

为考虑不同 VM 家族, 在式 (11) 中加入专门系数 μ_f 。属于家族 f 、具有 N 个 vCPU 的 VM, 其加速比估计为:

$$C(N, f) = \mu_f \cdot C(N). \quad (12)$$

观察本文三种 VM 家族的扩展曲线后发现, c5 家族使用 $\mu_f = 1$, c4 和 m5 家族使用 $\mu_f = 0.8$, 可以获得最佳拟合。

最后说明, 本文选择用 USL 估计任务执行时间的变化, 而未直接使用 AWS EC2 各 VM 类型上的具体测量值。因为这些测量值高度依赖 AWS EC2 实例的具体硬件; 采用 USL 则可使研究覆盖更多样的扩展性场景, 以及更大的数值集合。

5.1.4 调度算法

为进行比较, 除 EPOSS 外, 还考虑以下算法。首先选择第 2 节与第 4.1 节介绍的 HEFT 和 MOHEFT 作为基线, 并加入 HEFT 的变体 GreedyCost; 其唯一区别是最小化费用, 而非完工时间。虽然这些算法不面向概率形式化, 只考虑平均执行时间, 但可用于评估概率调度方法的优势。此外, 还考虑另外两种面向概率问题形式的先进算法: Dyna [13], 以及 [12] 提出的遗传方法, 下文记为 Genetic。这些算法已在第 2 节介绍。注意, 这些算法的作者决定不公开源码。因此, 我们依据作者论文提供的细节自行实现它们。

MOHEFT 与 EPOSS 的配置参数取 $K = 10$, 遵循 MOHEFT 作者在 [23] 中的建议; 并取 $\epsilon = 0.02$, 见第 4.2 节。对于 EPOSS 内的 Monte Carlo 仿真, 采用基于平均完工时间估计达到 95% 置信区间的统计停止准则。对于本文所有使用 Monte Carlo 仿真的算法, 观察到仿真对总执行时间影响较小, 通常不到 10%。

由于实验中的全部算法都能求解单目标形式, 即在工作流执行截止期下最小化费用, 见第 3 节, 因此先讨论相关结果。随后, 考虑到 Genetic 专门用于带约束多目标优化, 第 4.4 节定义了该问题, 再给出 EPOSS 与 Genetic 的专门比较。

5.1.5 输入参数配置

运行所有算法时改变多种输入参数配置。对于第一种问题形式, 一项配置对应参数 Θ 、 d 、 p_T 、 M^R 、 M_i^{type} 的一组取值, 其中 $\forall i \in \Theta$ 。我们将 Θ 设为第 5.1.3 节所述 21 种 VM 类型的以下不同子集:

- $\Theta_2 = \{c4.large, c4.xlarge\}$
- $\Theta_4 = \{c4.large, c4.xlarge, c4.2xlarge, c4.4xlarge\}$
- $\Theta_5 = \{\text{全部 c4 VM 类型}\}$
- $\Theta_{13} = \{\text{全部 c4 与 m5 VM 类型}\}$
- $\Theta_{21} = \{\text{全部 c4、c5 与 m5 VM 类型}\}$

其中, Θ_x 的下标 x 表示集合中的 VM 类型数。根据每个工作流的计算需求改变截止期 d 。具体而言, 对 CyberShake、Epigenomics 和 LIGO, 设 d 为 900 秒; 对 Montage 为 2400 秒, 对 SIPHT 为 1800 秒。 p_T 取三个数值: 0.75、0.9 和 0.95, 分别对应第 75、90 和 95 百分位。对于最大资源量相关参数

1. <https://aws.amazon.com/ec2/instance-types/>

M^R 和 M_i^{type} , Genetic 不考虑这类约束, Dyna 也只部分考虑, 见表 1。因此, 为公平比较, 先报告移除这些约束后的结果, 随后再加入它们。

对于带约束多目标形式, 一项配置还包括参数 c 与 p_C 。该形式要求费用以 p_C 的概率不超过 c , 见第 4.4 节。取 $c = 10$ 美元, $p_C = 0.9$ 。

最后, 由于 EPOSS、Dyna 和 Genetic 受 Monte Carlo 仿真随机性影响, 对每项配置运行 10 次, 并对结果取平均; 每次使用不同的随机数生成器初始种子。

5.1.6 解的评估

我们按照既定任务分配计划仿真 workflow 执行, 评估算法找到的解。具体而言, 给定算法为某项输入配置确定的解 s , 执行 10,000 次仿真; 每次从各任务对应分布随机抽取执行时间。再计算完工时间不超过截止期 d 的仿真比例, 得到解 s 的概率 $P(T(s) \leq d)$ 估计值, 将其与目标概率 p_T 比较, 验证解 s 是否满足式 (2) 的约束。类似地, 对于带约束多目标问题, 也通过仿真评估概率 $P(C(s) \leq c)$, 与目标概率 p_C 比较, 验证解 s 是否同时满足式 (8) 的约束。还根据仿真结果计算解 s 的平均完工时间与平均费用; 不同问题形式要求最小化其中相应指标。

5.1.7 算法执行时间比较

为比较调度算法的执行时间, 在同一台机器上运行全部算法, 具体为具有 8 个 vCPU、16 GB 内存、Ubuntu 22.04 的 EC2 c5.2xlarge 实例。所报告时间为每种算法 10 次执行的平均值。

5.2 单目标形式下的结果

本节结果对应单目标问题形式。对 Genetic 与 Dyna, 分别报告 50,000 次内部迭代, 即作者原实验所用次数, 以及 100,000 次内部迭代的结果, 记为 Genetic-50k、Genetic-100k 和 Dyna-50k、Dyna-100k。对 EPOSS, 报告顺序实现以及第 4.3 节并行实现 P-EPOSS 的结果。

本节先给出汇总结果, 对全部配置、全部 p_T 值, 即 0.75、0.9 和 0.95, 以及全部五个工作流的结果取平均。重点考虑执行时间按扩展性场景 A 计算、且服从 Gamma 分布的情况。之后, 报告不同 p_T 值和不同工作流的详细结果; 最后, 讨论其他扩展性场景及其他任务执行时间分布。

5.2.1 汇总结果

图 3 给出扩展性场景 A、任务执行时间服从 Gamma 分布时的汇总结果。上图表示算法找到可行解的配置比例, 即算法能够找到满足目标截止期概率之解的配置占比; 下图表示所得解的费用, 单位为美元。结合图 3 的数据, 以及表 4 报告的各项算法平均执行时间, 进行分析。

首先, 图 3 的上图显示, HEFT 总能找到可行解。HEFT 忽略费用, 只寻找完工时间最短的解, 因此预期它在全部算法中能找到满足截止期概率最高的解, 但费用也较高。图 3 下图的结果证实了这一点。GreedyCost 的行为相反: 它忽略完工时间, 尽量最小化费用。数据表明, GreedyCost 平均找到全部算法中费用最低的解; 但仅在 VM 类型集合较小时, 其满足截止期的配置比例才高于 0%。

MOHEFT 的结果体现了完工时间与费用之间的权衡, 位于 HEFT 与 GreedyCost 之间。但其可行解比例随 VM 类型数增加而下降, 在 8 种或更多类型时降至 0%。这表明, 当 VM 类型数不小, 仅依赖平均执行时间的 MOHEFT 一类算法, 就会失去为概率问题形式找到可行解的能力。保证 workflow 平均执行时间满足截止期的解, 不一定保证其第 75 百分位或更高百分位也满足截止期。另一个观察是, 前述三种算法的执

表 4: 不同算法在不同 VM 类型集合下的平均执行时间

算法	执行时间 (秒)				
	Θ_2	Θ_4	Θ_8	Θ_{13}	Θ_{21}
HEFT	0.01	0.01	0.02	0.04	0.08
MOHEFT	0.12	0.59	2.64	7.96	15.85
GreedyCost	0.01	0.01	0.02	0.04	0.08
Dyna-50k	517.95	3550.19	3600.58	3600.66	3600.06
Dyna-100k	750.77	3600.22	3600.62	3600.60	3600.16
Genetic-50k	1234.56	1252.84	1807.25	1791.15	1777.75
Genetic-100k	2599.34	2647.30	3340.96	3466.74	3457.59
EPOSS	2.97	9.62	28.26	42.12	81.86
P-EPOSS	1.58	5.78	16.11	28.34	61.19

行时间通常低于其他算法, 见表 4; 最坏情况下, MOHEFT 约 15 秒完成。但总体结果仍表明, 它们通常无法充分处理该问题: 部分情况找到的解费用很高, 其余情况则仅有较小比例的解可行。

对于其他算法, 首先观察 Dyna 的执行时间: 在 Θ_2 下就不低, 从 Θ_2 扩展到 Θ_4 后, 还会增加 5–6 倍。Dyna 作者的研究 [13] 只报告了包含 4 种 VM 类型的实验, 无法观察更大类型数量对算法执行时间的影响。本文实现的 Dyna, 在 8 种类型下需要超过一小时才能终止; 超过 8 种后, 完成时间进一步增加。其他算法则即使在最大的 θ , 即 21 种 VM 类型下, 也至多约一小时完成。

我们发现, Dyna 执行时间快速增长的主要原因与其搜索策略有关: 按 VM 类型成本递增探索。在作者 [13] 第 5 页的算法 1 中, 这发生在从行 6 开始的 *while* 循环: 寻找满足截止期的解, 并在需要时更新上界与所选解, 见行 12–14。该循环中, VM 类型集合增大时, 探索空间呈指数增长, 即使算法尝试剪去部分搜索树路径, 也难以避免搜索时间快速增加。由于耗时较长, 若对全部含 8 种或更多 VM 类型的配置完整运行 Dyna, 将需要过量时间。因此, 为对本文全部算法采用公平比较标准, 若一个算法运行一小时后仍未结束, 就停止执行, 并以强制终止时已经找到的最佳配置作为结果。表 4 的斜体数字标出这些情况。由此, Dyna 在 8 种或更多 VM 类型下返回的配置, 常无法满足问题形式中的约束; 特别是 13 种或更多类型时, 从未找到可行解。因此, 为简洁起见, 后续数据表省略 Dyna 在 13 种或更多类型下的结果。

除上述 Dyna 问题外, 结果还表明, 对所有 VM 类型集合规模, EPOSS 找到可行解的配置比例通常高于 Dyna 和 Genetic。Genetic 在可行解比例上优于 Dyna; 费用方面也如此, 除了类型数达到 8 或更多的情况。但这些情况下 Dyna 找到的解始终不可行, 因此费用更低有其原因: Dyna 从低费用类型开始搜索。另一个观察是, Genetic 从 50,000 次迭代增加到 100,000 次后, 通常没有改善或改善很小; 只有 θ_{21} 下可行解比例明显提高, 但费用也有不可忽略的增加, 而且所有情况下执行时间都几乎翻倍。因此, 100,000 次迭代相比 50,000 次没有明显优势。EPOSS 的费用低于 Dyna 和 Genetic, 也清楚展示其相对 HEFT 的优势: HEFT 虽然总能找到可行解, 但这些解的费用是 EPOSS 解的两到五倍。

表 4 展示 EPOSS 的另一个明显优势: 采用 50,000 次迭代的 Dyna 与 Genetic, 执行时间比 EPOSS 高一到三个数量级, 具体为 21 至 875 倍。这实现了 EPOSS 的主要目标, 证明其效率高于其他先进概率调度算法。数据还显示, 并行实现 P-EPOSS 能进一步缩短时间。EPOSS 与 P-EPOSS 在同一机器上运行, 但 P-EPOSS 使用 8 个并行线程, 利用全部 8 个 vCPU; 顺序版 EPOSS 则仅利用一个 vCPU。在解的可行性与费用方面, P-EPOSS 通常与 EPOSS 相近, 但执行时间进一步降低。这一降幅并非各情况都相同, 全部执行的平均加速比为 1.47。如第 4.3 节估计, 该情形的最大理论加速比为 $\log_2 8 = 3$ 。P-EPOSS 仍有无法并行的代码块, 实际硬件也存

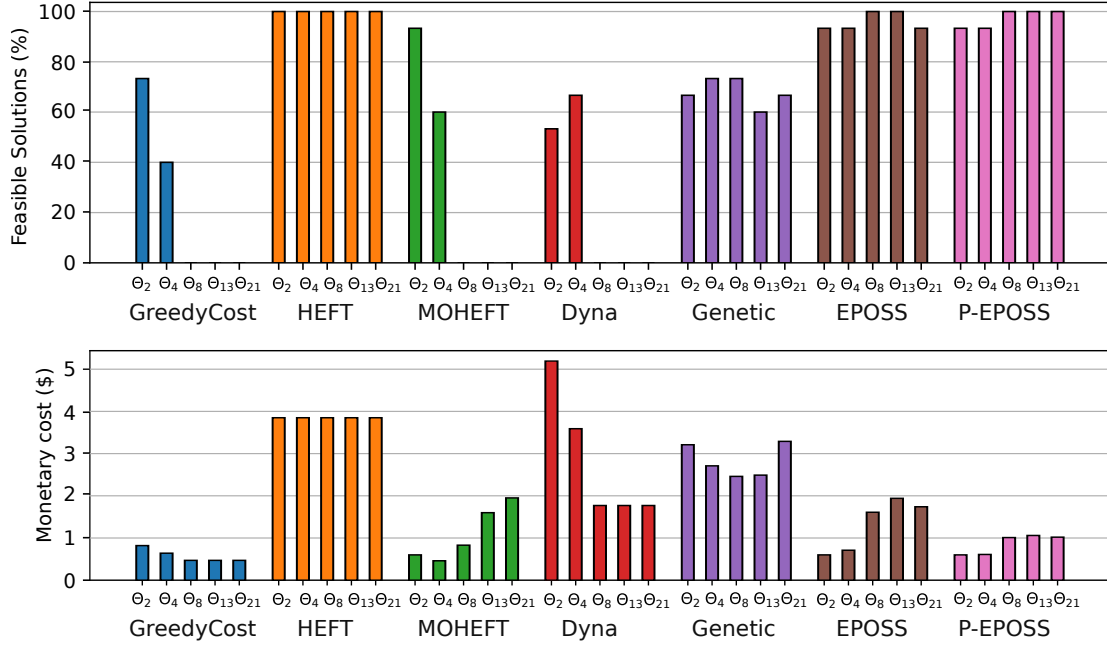


图 3: 不同算法与不同 VM 类型集合的汇总结果

图中术语: Feasible Solutions (%) 为可行调度方案比例; Monetary cost (\$) 为货币费用 (美元); 横轴中的数字表示 VM 类型集合的大小, 算法名称保持原样。

在共享资源, 因此平均加速比为 1.47 是可以预期的。

5.2.2 详细结果

为更深入展示实验结果, 本小节按单个工作流, 以及三个不同 p_T 值, 分别报告数据。由于篇幅限制, 重点考虑 Θ_8 、 Θ_{13} 和 Θ_{21} , 这些情况更有利于评估本文调度算法。此外, 省略 Dyna 在超过 8 种 VM 类型时的结果, 因为如前所述, 它在这些设置下无法找到可行解。同样由于篇幅限制, 正文只报告 Epigenomics, 其余工作流的结果列于本文补充材料。

表 5 报告 Epigenomics 在 p_T 为 0.75、0.9 和 0.95 时的数据。每行对应一个算法在一项配置下的结果。具体而言, 给定算法返回的解 s , 命中率列显示按照第 5.1.6 节方法评估解 s 所得结果; 更明确地, 它报告解 s 的完工时间低于所需截止期的比例, 即满足 $T(s) \leq d$ 的次数比例。如果该比例小于等于 p_T , 则解 s 可接受, 即满足问题形式中的约束 $P(T(s) \leq d) < p_T$ 。² 命中率列中满足这一要求的数值加粗。费用列显示所得解的平均费用, 单位为美元; 最后, 执行时间 (秒) 列报告算法执行时间。

对于 p_T 为 0.9 的情况, 图 4 进一步展示完工时间、费用与算法执行时间。完工时间和费用的分布, 用从第一四分位数到第三四分位数的箱体表示; 须线从第 5th 百分位延伸到第 95th 百分位, 水平线表示中位数。绿色箱体表示产生可行执行的调度解, 即满足 $P(T(s) \leq d) \geq 0.9$; 灰色条表示不可行解。红色虚线标出选定截止期, 实验中为 900 秒。

分析表 5 与图 4 可见, 总体趋势与表 4 汇总数据一致, 没有发现明显偏离既有观察的特殊情况。其他工作流的数据也支持这一点, 见补充材料。因此, EPOSS 在不同工作流与设置中的行为, 类似于汇总数据所呈现的行为。我们认为这是积极结果, 因为它确认 EPOSS 的收益通常并不偏向特定工作流。

5.2.3 不同任务执行时间分布与扩展性场景

为验证不同任务执行时间分布与扩展性场景是否影响算法结果, 再分析一些汇总数据。表 6 给出除 Gamma 外的其他分布, 即确定性时长、半正态分布和均匀分布的结果, 配置为 Θ_8 、 $p_T = 0.9$ 。如预期, 在确定性执行时间下, 由于工作流全部任务都不存在时长变化, 所有算法找到可行解的比例均更高。半正态分布得到类似结果。均匀分布则更具挑战, 各算法的可行解比例都下降。具体而言, HEFT、MOHEFT 与 EPOSS 在确定性时长和半正态分布下总能找到可行解, Genetic 则有 93.33% 的情况找到可行解。³ 但 EPOSS 和 MOHEFT 找到的解平均费用更低。均匀分布下, EPOSS 优于 MOHEFT; 不过这也是 EPOSS 最困难的分布, 其可行解比例降至 73%, 平均费用增加到 3.52。注意, 均匀分布代表一种极端情况: 时长被假设在最小与最大值之间均匀随机分布, 降低了任务执行时间的可预测性。这在真实环境中通常较少观察到, 例如参见 [40], 因为执行时间通常并非均匀分布, 而更集中在均值附近。

表 7 给出第 5.1.3 节扩展性场景 A、B、C 的汇总结果, 采用 $p = 0.9$ 和 Gamma 分布。场景 A 是第 5.2.1 节既有结果的汇总视图, 确认此前观察。场景 B 更简单, 加速比线性增长, 数据与 A 相近。场景 C 则更复杂: 并行度超过某个水平后, 加速比下降, 因此所有算法的结果通常都更差。总体上, HEFT 在不同场景中具有最高可行解比例, 但在 A、B 中平均费用也最高。在这两种场景中, EPOSS 的可行解比例达到 93.33, 平均费用低于其他算法。在更困难的 C 中, HEFT 与 EPOSS 获得最高可行解比例; EPOSS 在这一比例上也优于 MOHEFT 和 Genetic, 但平均费用更高。我们认为, 原因是某个并行度水平之后加速比下降, 使执行时间可预测性减弱。不过, 如第 5.1.3 节所述, 场景 C 在实际环境中较少出现, 科学工作流尤其如此。

2. 译者注: 原文此句的不等号方向与前文概率截止期约束相反。前文要求按时完成概率不小于目标概率, 而此句却写成小于或等于; 表格中加粗的是高于目标的命中率。此处保留原文表达, 供对照核查。

3. 译者注: 原文这段对半正态分布的描述与表中数据不一致。表中 HEFT、MOHEFT、Genetic 和 EPOSS 的对应可行解比例分别为 86.67%、6.67%、53.34% 和 80.00%。原表和正文说法均保留。

表 5: Epigenomics 工作流上各调度算法的比较。

VM 集合	算法	$p_T = 0.75$			$p_T = 0.9$			$p_T = 0.95$		
		命中率 (%)	费用 (美元)	执行时间 (秒)	命中率 (%)	费用 (美元)	执行时间 (秒)	命中率 (%)	费用 (美元)	执行时间 (秒)
Θ_8	HEFT	100.0	0.69	0.02	100.0	0.69	0.02	100.0	0.69	0.02
	MOHEFT	64.5	0.18	0.85	64.5	0.18	0.85	64.5	0.18	0.85
	GreedyCost	0.1	0.13	0.02	0.1	0.13	0.02	0.1	0.13	0.02
	Dyna-50k	0.2	0.48	3600.35	0.2	0.48	3600.26	0.2	0.48	3600.33
	Dyna-100k	0.2	0.48	3600.41	0.2	0.48	3600.28	0.2	0.48	3600.31
	Genetic-50k	84.4	0.78	1033.19	94.2	1.27	1042.62	97.9	1.52	1020.14
	Genetic-100k	78.0	0.55	2100.38	100.0	1.17	2094.36	99.9	1.25	2064.84
	EPOSS	96.0	0.21	10.82	96.0	0.21	10.84	100.0	0.24	10.33
	P-EPOSS	95.2	0.26	4.06	95.2	0.26	4.08	96.6	0.29	4.15
Θ_{13}	HEFT	100.0	0.69	0.03	100.0	0.69	0.03	100.0	0.69	0.03
	MOHEFT	64.5	0.18	2.40	64.5	0.18	2.41	64.5	0.18	2.39
	GreedyCost	0.1	0.13	0.03	0.1	0.13	0.03	0.1	0.13	0.03
	Genetic-50k	82.4	0.75	1057.26	89.8	0.74	1046.87	99.5	1.23	1041.67
	Genetic-100k	97.7	1.33	2088.53	87.3	0.58	2067.25	90.0	0.59	2108.94
	EPOSS	96.0	0.21	16.20	96.0	0.21	16.12	99.2	0.35	15.51
	P-EPOSS	95.2	0.26	10.13	95.2	0.26	10.09	96.6	0.29	10.21
Θ_{21}	HEFT	100.0	0.69	0.05	100.0	0.69	0.05	100.0	0.69	0.05
	MOHEFT	64.5	0.18	8.51	64.5	0.18	8.45	64.5	0.18	8.47
	GreedyCost	0.1	0.13	0.05	0.1	0.13	0.05	0.1	0.13	0.05
	Genetic-50k	90.8	1.48	1055.65	90.8	1.51	1053.54	91.5	1.10	1051.30
	Genetic-100k	77.1	0.67	2125.96	88.7	0.70	2115.45	96.2	0.90	2123.56
	EPOSS	96.0	0.21	26.68	96.0	0.21	26.64	97.4	0.22	24.87
	P-EPOSS	95.2	0.26	17.15	95.2	0.26	17.24	95.2	0.26	17.05

表 6: 不同任务执行时间分布下的结果。

分布	算法	可行解比例 (%)	费用 (美元)
确定性时长	HEFT	100.00	2.93
	MOHEFT	100.00	0.78
	Genetic	93.33	1.91
	EPOSS	100.00	0.78
半正态分布	HEFT	86.67	2.89
	MOHEFT	6.67	1.03
	Genetic	53.34	2.51
	EPOSS	80.00	2.22
均匀分布	HEFT	86.67	2.90
	MOHEFT	26.67	1.06
	Genetic	46.67	3.30
	EPOSS	73.33	3.52

表 7: 不同扩展性场景下的结果。

场景	算法	可行解比例 (%)	费用 (美元)
A	HEFT	100.00	2.65
	MOHEFT	80.00	0.59
	Genetic	80.00	1.63
	EPOSS	93.33	0.54
B	HEFT	100.00	3.94
	MOHEFT	73.33	0.90
	Genetic	86.67	1.86
	EPOSS	93.33	0.78
C	HEFT	86.67	2.16
	MOHEFT	73.33	1.13
	Genetic	66.67	3.64
	EPOSS	86.67	3.75

5.2.4 资源量限制

如前所述, 为公平比较, 之前的实验结果移除了问题形式中关于 N^R 和 N_i^{type} 的约束。本小节报告加入这些约束后的部分结果。注意, 工作流执行期间若某类资源已达到数量上限, 再请求同类新资源时, 必须等待已有同类资源释放, 这会增加完工时间。EPOSS 能完整考虑资源限制, 因此可能比忽略这些

表 8: 提供商容量受限的结果, 容量用可并发分配的最大 vCPU 数表示。缺失结果表示算法无法在给定约束下确定可行调度。

M^R	算法	费用 (美元)	
		$d=5400$ 秒	$d=7200$ 秒
25	Genetic	—	—
	HEFT	—	—
	MOHEFT	—	—
	EPOSS	—	1.16
50	Genetic	—	—
	HEFT	—	—
	MOHEFT	—	—
	EPOSS	0.93	1.13
100	Genetic	1.68	3.30
	HEFT	—	—
	MOHEFT	—	—
	EPOSS	1.20	1.16
400	Genetic	1.61	2.46
	HEFT	11.86	19.45
	MOHEFT	—	—
	EPOSS	1.31	1.16

限制的 HEFT、MOHEFT 和 Genetic 找到更好的解。表 8 给出上述算法在不同限制设置下所得解的平均费用。具体而言, 可同时使用的最大 vCPU 数 M^R 取 25、50、100、400; 每种类型的 VM 最大数量 N_i^{type} 对每个 $i \in \theta$ 均取 10; 截止期 d 取 5400 秒与 7200 秒。数据对应 θ_8 下的 Epigenomics 任务时长服从 Gamma 分布。表 8 中的横线表示算法返回解不可行, 因为未满足约束 $P(T(s) \leq d) < p_T$ 。⁴截止期为 5400 秒时, $M^R = 25$ 下没有算法找到可行解; $M^R = 50$ 下, 只有 EPOSS 找到可行解。Genetic 仅在 $M^R = 100$ 和 $M^R = 400$ 时找到可行解, 且费用始终高于 EPOSS。采用更大的截止期

4. 译者注: 这里的概率不等号方向也与前文问题定义相反, 沿用原文。

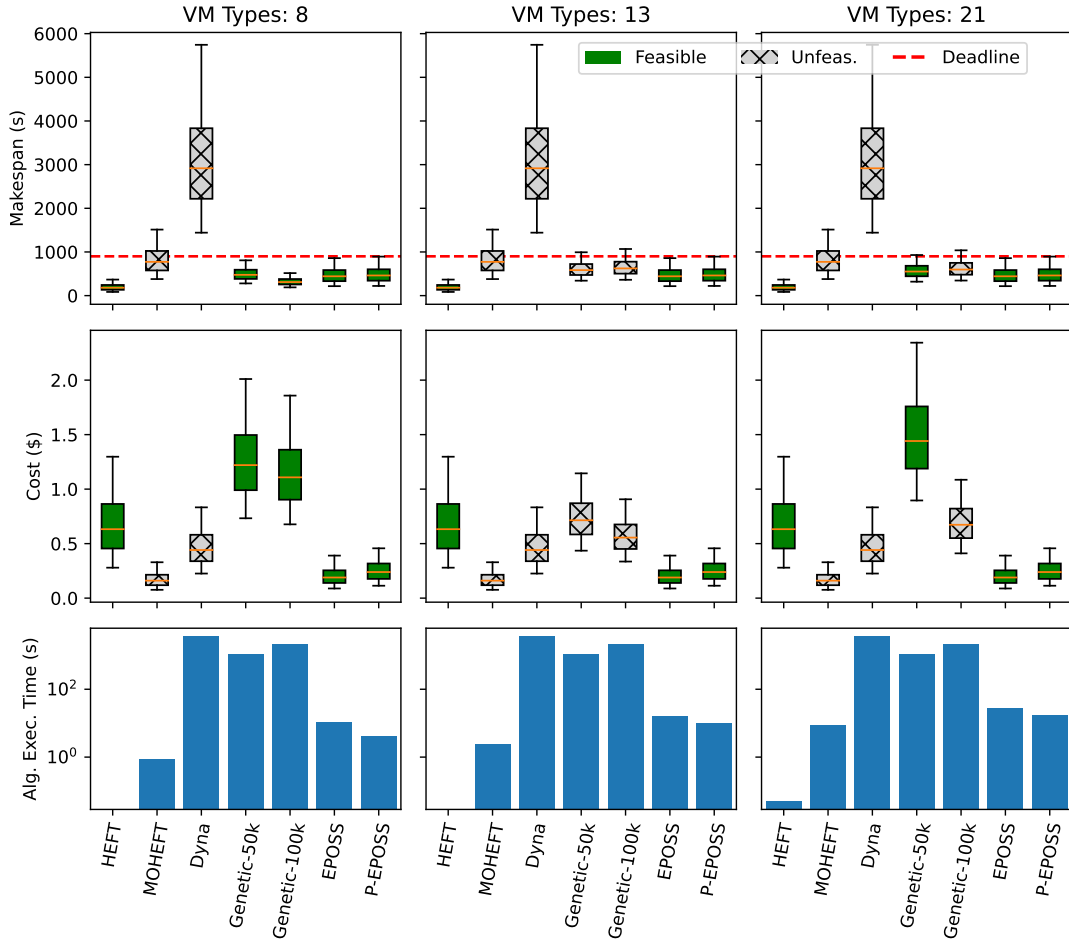


图 4: Epigenomics 工作流的完工时间、费用与算法执行时间图示 ($p_T = 0.9$)

图中术语: Makespan (s) 为总完工时间 (秒); VM Types 为 VM 类型数量; Cost (\$) 为费用 (美元); Alg. Exec. Time (s) 为算法运行时间 (秒); Feasible、Unfeas.、Deadline 分别表示可行、不可行和截止期。

时, EPOSS 即使在 $M^R = 25$ 下也能找到可行解; 其他算法则仅在 $M^R = 100$ 和 $M^R = 400$ 下找到。总体而言, 数据明确展示了 EPOSS 在某些情况下通过考虑资源限制所提供的优势。

5.3 多目标形式下的结果

实验所用文献算法中, 只有 Genetic 同时能够求解多目标问题并采用概率形式化。本小节比较第 4.4 节的 M-EPOSS 与 Genetic, 展示多目标结果。该形式同时最小化完工时间与费用, 因此需要寻找 Pareto 最优解集。图 5 给出图形比较, 展示全部工作流在 θ_8 、Gamma 分布条件下, M-EPOSS 与 Genetic 返回的 Pareto 前沿。其他情形的结果类似。这里展示 Genetic-100k 的结果, 因为它略好于 Genetic-50k。

为提供更清楚的定量比较, 表 9 还报告对应超体积, 它是评估多目标优化算法性能的参考指标 [41]。超体积对应前沿所支配解空间的大小, 因此越大越好; 图 6 给出示例。选定参考点, 是由前沿全部点中最大费用与最大完工时间构成的空间点。

表 9 以百分比比较 EPOSS、Genetic-50k 和 Genetic-100 所计算超体积, 以及算法执行时间。总体而言, EPOSS 与 Genetic 的前沿超体积十分接近, 差异从未超过 5%。⁵ 在五种工作流

表 9: 不同算法所计算前沿的相对超体积及对应执行时间。

作业	算法	超体积 (%)	算法执行时间 (秒)
Epigenomics	M-EPOSS	100.00	17.3
	Genetic-50	97.45	351.3
	Genetic-100	98.43	1015.8
SIPHT	M-EPOSS	100.00	73.7
	Genetic-50k	100.95	871.7
	Genetic-100k	101.16	2473.4
CyberShake	M-EPOSS	100.00	114.4
	Genetic-50k	100.84	570.6
	Genetic-100k	101.37	1480.6
Montage	M-EPOSS	100.00	110.0
	Genetic-50k	95.55	495.3
	Genetic-100k	96.34	1264.1
LIGO	M-EPOSS	100.00	231.9
	Genetic-50k	105.06	795.3
	Genetic-100k	105.64	1946.4

中, 没有一种方法始终胜过另一种: EPOSS 在 Epigenomics 和 Montage 上超体积略高, Genetic 在 SIPHT、CyberShake 和 LIGO 上略好。基本上, 两算法超体积没有明显差异。但执行时间差异很大: EPOSS 比 Genetic-50 缩短 70% 至 97%, 比 Genetic-100 缩短 88% 至 98%。因此, EPOSS 能提供与 Genetic 同样好的解, 同时大幅缩短执行时间。

5. 译者注: 表中 LIGO 的 Genetic-100k 相对超体积为 105.64%, 与以 100% 归一化的 M-EPOSS 相差 5.64%; 这里保留原文“从未超过 5%”的说法。

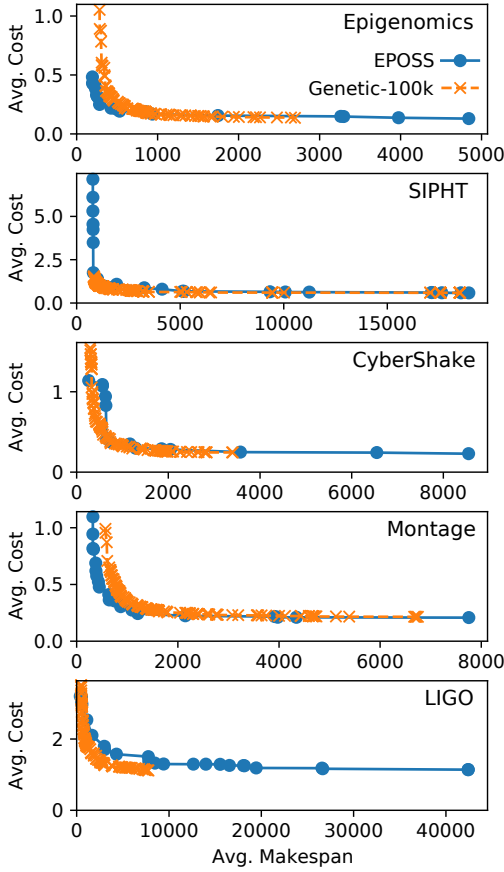


图 5: EPOSS 与 Genetic 计算得到的 Pareto 前沿。
图中术语: Avg. Makespan 为平均总完工时间; Avg. Cost 为平均费用。

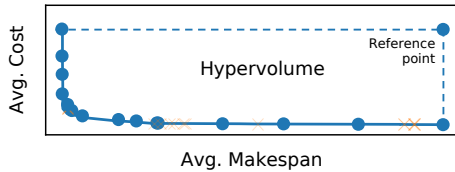


图 6: Pareto 前沿的超体积示例。
图中术语: Avg. Makespan 为平均总完工时间; Avg. Cost 为平均费用; Hypervolume 为超体积; Reference point 为参考点。

5.4 随 workflow 规模的扩展性

最后, 比较本文算法与 Genetic 随 workflow 规模变化的扩展性。Genetic 的数据取自作者 [12] 提供的扩展性结果, 对应任务数从 24 增至 997 的 Epigenomics。为比较扩展能力, 各算法执行时间都以其在最小 workflow, 即 24 个任务下的执行时间归一化。注意, 本文实验已显示 Genetic 的执行时间是 EPOSS 的 21 至 875 倍, 因此本节数据只应用于比较扩展能力, 而非直接比较性能。图 7 比较 Genetic、EPOSS 与 P-EPOSS 的扩展曲线。可以明显看到, 从约 80 个任务开始, Genetic 的增长快于 EPOSS。EPOSS 的并行实现似乎能够保持线性扩展直到 1000 个节点。事实上, 观察到从 150 个任务开始, P-EPOSS 曲线上相邻点的增量比值稳定在约 1.61。

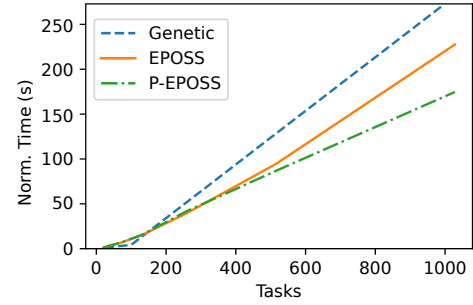


图 7: Genetic、EPOSS 和 P-EPOSS 的归一化算法执行时间随 workflow 规模的变化。

图中术语: Tasks 为任务数量; Norm. Time (s) 为归一化运行时间; 算法名称保持原样。

6 结论

本文提出面向 IaaS 云的工作流调度算法 EPOSS。它以 DAG 调度问题的概率形式化处理云资源性能不确定性, 并引入约束以描述 IaaS 提供商的实际要求。

因此, 与大多数基于确定性执行时间的现有方案不同, EPOSS 能够求解更复杂、计算成本更高的概率问题, 同时保持总体计算成本可接受, 且远低于先进概率算法。本文还提出 EPOSS 并行版本, 利用硬件并行性进一步减少求解时间。

广泛评估显示, EPOSS 在多种配置场景中均高效且有效, 包括不同 workflow、VM 集合、任务执行时间分布、扩展性曲线, 以及不同可用资源约束。具体而言, 结果表明, 在不损失解质量的情况下, EPOSS 相对其他概率算法可将求解时间缩短 10 倍至超过 100 倍。这实现了 EPOSS 的主要目标; 它被本文认为是当时最有效率的概率算法, 可用于广泛情境。

未来将扩展研究, 加入自动任务执行时间剖析与分布拟合策略, 设计能够在 IaaS 云上完全自动执行并优化工作流的框架。

致谢

本文工作获得 POR FESR Lazio 2014-2020 的“Progetti di Gruppi di Ricerca 2020”项目部分资助, 项目编号 A0375-2020-36556。

参考文献

- [1] G. Juve, A. L. Chervenak, E. Deelman, S. Bharathi, G. Mehta, and K. Vahi, “Characterizing and profiling scientific workflows,” *Future Gener. Comput. Syst.*, vol. 29, no. 3, pp. 682–692, 2013.
- [2] M. A. Rodriguez and R. Buyya, “A taxonomy and survey on scheduling algorithms for scientific workflows in IaaS cloud computing environments,” *Concurr. Comput. Pract. Exp.*, vol. 29, no. 8, p. e4041, 2017.
- [3] Y.-K. Kwok and I. Ahmad, “Static scheduling algorithms for allocating directed task graphs to multiprocessors,” *ACM Comput. Surv.*, vol. 31, no. 4, p. 406–471, 1999.
- [4] S. Zhao, X. Dai, I. Bate, A. Burns, and W. Chang, “DAG scheduling and analysis on multiprocessor systems: Exploitation of parallelism and dependency,” in *2020 IEEE Real-Time Systems Symposium*, 2020, pp. 128–140.
- [5] H. F. Sheikh, I. Ahmad, and D. Fan, “An evolutionary technique for performance-energy-temperature optimized scheduling of parallel tasks on multi-core processors,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 3, pp. 668–681, 2016.
- [6] E. Hou, N. Ansari, and H. Ren, “A genetic algorithm for multiprocessor scheduling,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 5, no. 2, pp. 113–120, 1994.

- [7] A. Wu, H. Yu, S. Jin, K.-C. Lin, and G. Schiavone, "An incremental genetic algorithm approach to multiprocessor scheduling," *IEEE Trans. Parallel Distrib. Syst.*, vol. 15, no. 9, pp. 824–834, 2004.
- [8] F. Pop, C. Dobre, and V. Cristea, "Genetic algorithm for DAG scheduling in Grid environments," in *IEEE 5th Int'l Conf. on Intelligent Computer Communication and Processing*, 2009.
- [9] X. Tang, K. Li, G. Liao, K. Fang, and F. Wu, "A stochastic scheduling algorithm for precedence constrained tasks on Grid," *Future Gener. Comput. Syst.*, vol. 27, no. 8, pp. 1083–1091, 2011.
- [10] S. Selvi and D. Manimegalai, "DAG scheduling in heterogeneous computing and grid environments using variable neighborhood search algorithm," *Appl. Artif. Intell.*, vol. 31, no. 2, 2017.
- [11] A. Verma and S. Kaushal, "A hybrid multi-objective particle swarm optimization for scientific workflow scheduling," *Parallel Comput.*, vol. 62, pp. 1–19, 2017.
- [12] M. C. Calzarossa, M. L. Della Vedova, L. Massari, G. Nebbione, and D. Tessera, "Multi-objective optimization of deadline and budget-aware workflow scheduling in uncertain clouds," *IEEE Access*, vol. 9, pp. 89 891–89 905, 2021.
- [13] A. C. Zhou, B. He, and C. Liu, "Monetary cost optimizations for hosting workflow-as-a-service in iaas clouds," *IEEE Trans. Cloud Comput.*, vol. 4, no. 1, pp. 34–48, 2016.
- [14] H. Mezni, S. Aridhi, and A. Hadjali, "The uncertain cloud: State of the art and research challenges," *Int. J. Approx. Reason.*, vol. 103, pp. 139–151, 2018.
- [15] W. Lin, C. Xiong, W. Wu, F. Shi, K. Li, and M. Xu, "Performance interference of virtual machines: A survey," *ACM Comput. Surv.*, vol. 55, no. 12, mar 2023. [Online]. Available: <https://doi.org/10.1145/3573009>
- [16] S. Ristov, R. Mathá, and R. Prodan, "Analysing the performance instability correlation with various workflow and cloud parameters," in *25th Euromicro Int'l Conf. on Parallel, Distributed and Network-based Processing*, 2017, pp. 446–453.
- [17] J. Schad, J. Dittrich, and J.-A. Quiané-Ruiz, "Runtime measurements in the cloud: Observing, analyzing, and reducing variance," *Proc. VLDB Endow.*, vol. 3, no. 1–2, p. 460–471, 2010.
- [18] K. R. Jackson, L. Ramakrishnan, K. Muriki, S. Canon, S. Cholia *et al.*, "Performance analysis of high performance computing applications on the amazon web services cloud," in *IEEE 2nd Int'l Conf. on Cloud Computing Technology and Science*, 2010, pp. 159–168.
- [19] Y. Yuan, H. Wang, D. Wang, and J. Liu, "On interference-aware provisioning for cloud-based big data processing," in *2013 IEEE/ACM 21st International Symposium on Quality of Service (IWQoS)*, 2013, pp. 1–6.
- [20] AWS, "AWS service quotas," https://docs.aws.amazon.com/general/latest/gr/aws_service_limits.html, 2023.
- [21] Google Cloud, "Allocation quotas," <https://cloud.google.com/compute/resource-usage>, 2023.
- [22] Microsoft Azure, "Azure subscription and service limits, quotas, and constraints," <https://learn.microsoft.com/en-us/azure/azure-resource-manager/management/azure-subscription-service-limits>, 2023.
- [23] J. J. D. Barrionuevo, H. M. Fard, and R. Prodan, "MOHEFT: A multi-objective list-based method for workflow scheduling," in *4th IEEE Int'l Conf. on Cloud Computing Technology and Science*, 2012, pp. 185–192.
- [24] S. Abrishami, M. Naghibzadeh, and D. Epema, "Deadline-constrained workflow scheduling algorithms for Infrastructure as a Service clouds," *Future Gener. Comput. Syst.*, vol. 29, no. 1, pp. 158–169, 2013.
- [25] Z. Zhu and X. Tang, "Deadline-constrained workflow scheduling in IaaS clouds with multi-resource packing," *Future Gener. Comput. Syst.*, vol. 101, pp. 880–893, 2019.
- [26] N. Anwar and H. Deng, "Elastic scheduling of scientific workflows under deadline constraints in cloud computing environments," *Future Internet*, vol. 10, no. 1, 2018.
- [27] H. M. Fard, S. Ristov, and R. Prodan, "Handling the uncertainty in resource performance for executing workflow applications in clouds," in *9th Int'l Conf. on Utility and Cloud Computing*, 2016.
- [28] H. Hu, Z. Li, H. Hu, J. Chen, J. Ge, C. Li, and V. Chang, "Multi-objective scheduling for scientific workflow in multicloud environment," *J. Netw. Comput. Appl.*, vol. 114, pp. 108–122, 2018.
- [29] W. Li, Y. Xia, M. Zhou, X. Sun, and Q. Zhu, "Fluctuation-aware and predictive workflow scheduling in cost-effective infrastructure-as-a-service clouds," *IEEE Access*, vol. 6, pp. 61 488–61 502, 2018.
- [30] L. Liu, M. Zhang, R. Buyya, and Q. Fan, "Deadline-constrained coevolutionary genetic algorithm for scientific workflow scheduling in cloud computing," *Concurr. Comput. Pract. Exp.*, vol. 29, no. 5, p. e3942, 2017.
- [31] J. Meena, M. Kumar, and M. Vardhan, "Cost effective genetic algorithm for workflow scheduling in cloud under deadline constraint," *IEEE Access*, vol. 4, pp. 5065–5082, 2016.
- [32] M. Rodríguez and R. Buyya, "Deadline based resource provisioning and scheduling algorithm for scientific workflows on clouds," *IEEE Trans. Cloud Comput.*, vol. 2, pp. 222–235, 2014.
- [33] P. Ngatchou, A. Zarei, and A. El-Sharkawi, "Pareto multi objective optimization," in *13th Int'l Conf. on Intelligent Systems Application to Power Systems*, 2005, pp. 84–91.
- [34] S. V. Kovalchuk, P. A. Smirnov, S. V. Maryin, T. N. Tchurov, and V. A. Karbovskiy, "Deadline-driven resource management within urgent computing cyberinfrastructure," *Procedia Computer Science*, vol. 18, pp. 2203–2212, 2013.
- [35] J. J. Durillo, R. Prodan, and J. G. Barbosa, "Pareto tradeoff scheduling of workflows on federated commercial clouds," *Simul. Model. Pract. Theory*, vol. 58, pp. 95–111, 2015.
- [36] H. Topcuoglu, S. Hariri, and M. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, 2002.
- [37] K. Deb, S. Agrawal, A. Pratap, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, 2002.
- [38] H. Chen, X. Zhu, G. Liu, and W. Pedrycz, "Uncertainty-aware online scheduling for real-time workflows in cloud service environment," *IEEE Trans. Serv. Comput.*, vol. 14, no. 4, pp. 1167–1178, 2021.
- [39] Y. Gao, L.-C. Canon, Y. Robert, and F. Vivien, "Scheduling independent stochastic tasks on heterogeneous cloud platforms," in *2019 IEEE Int'l Conf. on Cluster Computing*, 2019, pp. 1–11.
- [40] A. M. Chirkin, A. S. Z. Belloum, S. V. Kovalchuk, and M. X. Makkes, "Execution time estimation for workflow scheduling," in *2014 9th Workshop on Workflows in Support of Large-Scale Science*, 2014, pp. 1–10.
- [41] E. Zitzler, J. Knowles, and L. Thiele, "Quality assessment of pareto set approximations," in *Multiobjective Optimization: Interactive and Evolutionary Approaches*. Springer, 2008, pp. 373–404.

补充材料

以下各页提供论文《面向 IaaS 云的高效概率 workflow 调度》的补充材料，包括正文第 5.2.2 节所提及的其他实验数据。

表 1-4 给出了 CyberShake、LIGO、Montage 和 SIPHT 工作流在 p_T 分别等于 0.75、0.9 和 0.95 时的详细结果。需要说明的是，命中率列报告算法所得调度方案的总完工时间低于目标截止期的次数百分比。如果这一百分比小于或等于 p_T ，则调度方案 s 是可接受的，即满足问题定义中的约束 $P(T(s) \leq d) < p_T$ 。⁶在命中率列中，满足这一要求的数值以粗体标出。费用列给出所得方案的平均货币费用，单位为美元。最后，执行时间（秒）列报告调度算法的运行时间。

图 1-4 展示了总完工时间、货币费用和调度算法运行时间。具体而言，总完工时间和费用采用箱线图表示其分布：箱体从第一四分位数延伸到第三四分位数，须线从第 5th 百分位数延伸到第 95th 百分位数，箱内的水平线表示中位数。绿色箱体对应产生可行执行结果的调度方案，即满足 $P(T(s) \leq d) \geq 0.9$ 的方案；灰色柱体对应不可行方案。红色虚线标出所选截止期。

6. 译者注：本句按原文保留。原文这里关于“小于或等于”和公式严格小于号的表述，与正文正式定义中要求按时完成概率至少达到目标值的约束方向相反；应结合正文的问题定义及表中粗体数据核查。

表 1: CyberShake 工作流的实验结果。

VM 集合	算法	$p_T = 0.75$			$p_T = 0.9$			$p_T = 0.95$		
		命中率 (%)	费用 (美元)	执行时间 (秒)	命中率 (%)	费用 (美元)	执行时间 (秒)	命中率 (%)	费用 (美元)	执行时间 (秒)
Θ_8	GreedyCost	0.0	0.23	0.03	0.0	0.23	0.03	0.0	0.23	0.03
	HEFT	100.0	1.91	0.03	100.0	1.91	0.03	100.0	1.91	0.03
	MOHEFT	8.7	0.57	3.15	8.7	0.57	3.13	8.7	0.57	3.17
	Dyna-50k	0.0	0.94	3600.34	0.0	0.94	3600.32	0.0	0.94	3600.14
	Dyna-100k	0.0	0.94	3600.19	0.0	0.94	3600.57	0.0	0.94	3600.58
	Genetic-50k	78.5	1.38	1531.85	91.1	1.60	1541.00	95.1	1.58	1530.66
	Genetic-100k	74.4	1.47	3223.00	100.0	2.28	3180.88	94.2	1.50	2986.10
	EPOSS	79.5	0.52	27.08	96.2	0.98	28.15	99.3	0.86	26.89
	P-EPOSS	97.5	0.76	19.63	98.9	0.98	19.65	97.1	0.69	19.49
Θ_{13}	GreedyCost	0.0	0.23	0.05	0.0	0.23	0.05	0.0	0.23	0.05
	HEFT	100.0	1.91	0.05	100.0	1.91	0.05	100.0	1.91	0.05
	MOHEFT	3.3	1.91	10.94	3.3	1.91	10.96	3.3	1.91	10.91
	Genetic-50k	87.3	2.11	1519.18	80.6	3.42	1506.21	97.0	2.02	1525.59
	Genetic-100k	99.7	2.98	3097.73	94.8	1.91	3034.91	99.0	2.25	3047.05
	EPOSS	90.8	0.61	47.28	93.5	1.46	45.29	99.3	1.11	46.74
	P-EPOSS	98.4	0.76	35.18	98.4	0.76	35.24	98.4	0.76	35.34
Θ_{21}	GreedyCost	0.0	0.23	0.10	0.0	0.23	0.10	0.0	0.23	0.10
	HEFT	100.0	1.91	0.10	100.0	1.91	0.10	100.0	1.91	0.10
	MOHEFT	16.3	0.55	21.22	16.3	0.55	21.37	16.3	0.55	21.35
	Genetic-50k	73.2	1.58	1551.17	85.6	1.76	1544.36	81.6	3.23	1538.05
	Genetic-100k	92.0	0.95	2996.63	66.0	6.71	3165.01	98.2	2.16	3018.90
	EPOSS	97.3	0.86	91.41	97.3	0.86	91.70	98.6	0.77	91.63
	P-EPOSS	96.8	0.60	76.08	96.8	0.60	76.28	96.8	0.60	75.66

表 2: LIGO 工作流的实验结果。

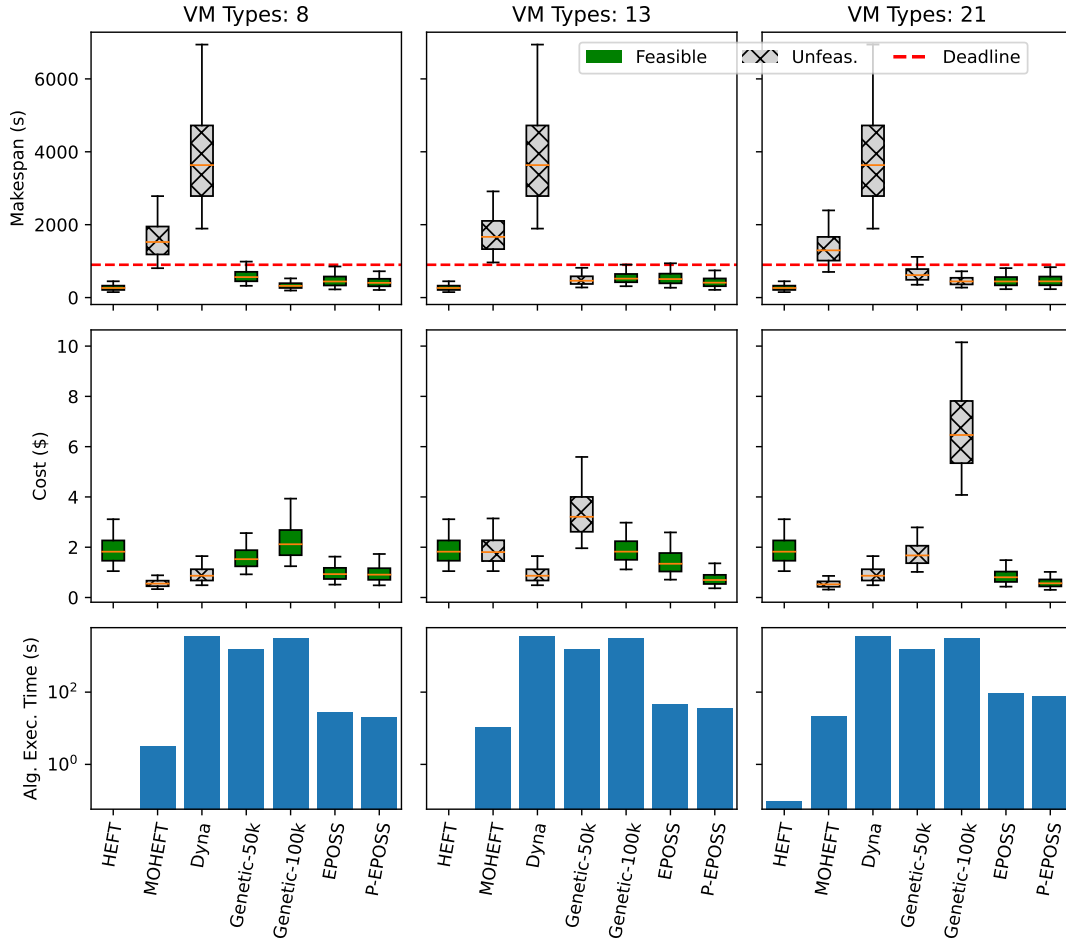
VM 集合	算法	$p_T = 0.75$			$p_T = 0.9$			$p_T = 0.95$		
		命中率 (%)	平均费用 (美元)	执行时间 (秒)	命中率 (%)	平均费用 (美元)	执行时间 (秒)	命中率 (%)	平均费用 (美元)	执行时间 (秒)
8	GreedyCost	0.0	1.16	0.03	0.0	1.16	0.03	0.0	1.16	0.03
	HEFT	100.0	6.69	0.03	100.0	6.69	0.03	100.0	6.69	0.03
	MOHEFT	63.2	1.76	5.17	63.2	1.76	5.13	63.2	1.76	5.18
	Dyna-50k	0.0	1.80	3600.70	0.0	1.80	3600.15	0.0	1.80	3600.66
	Dyna-100k	0.0	1.80	3600.41	0.0	1.80	3600.36	0.0	1.80	3600.11
	Genetic-50k	77.0	2.69	2145.36	94.7	2.92	2124.81	97.8	5.99	2138.24
	Genetic-100k	98.9	7.85	4517.37	87.5	2.69	4534.59	95.8	2.64	4648.29
	EPOSS	90.9	2.04	57.94	99.5	2.24	57.67	99.5	2.24	57.39
	P-EPOSS	92.2	2.20	27.21	93.5	2.03	27.30	99.5	1.89	27.31
13	GreedyCost	0.0	1.16	0.05	0.0	1.16	0.05	0.0	1.16	0.05
	HEFT	100.0	6.69	0.05	100.0	6.69	0.05	100.0	6.69	0.05
	MOHEFT	8.8	3.67	12.91	8.8	3.67	12.85	8.8	3.67	12.84
	Genetic-50k	95.5	6.66	2116.25	90.6	3.11	2106.55	95.1	6.87	2080.25
	Genetic-100k	93.0	2.79	4389.57	88.3	2.44	4412.43	99.6	5.01	4299.46
	EPOSS	92.0	2.61	68.71	92.7	2.02	76.63	100.0	6.25	96.96
	P-EPOSS	89.9	2.25	46.27	92.9	2.71	46.38	99.9	2.30	46.52
21	GreedyCost	0.0	1.16	0.09	0.0	1.16	0.09	0.0	1.16	0.09
	HEFT	100.0	6.69	0.09	100.0	6.69	0.09	100.0	6.69	0.09
	MOHEFT	4.1	6.90	25.28	4.1	6.90	25.42	4.1	6.90	25.28
	Genetic-50k	72.4	3.12	2089.59	94.1	3.86	2029.39	94.7	3.15	2066.19
	Genetic-100k	99.8	9.28	4369.91	99.8	5.77	4342.21	97.5	4.05	4337.04
	EPOSS	84.1	2.34	141.77	94.4	2.25	146.98	94.4	2.25	146.93
	P-EPOSS	90.2	1.96	105.20	95.5	2.36	104.96	100.0	2.18	104.41

表 3: Montage 工作流的实验结果。

VM 集合	算法	$p_T = 0.75$			$p_T = 0.9$			$p_T = 0.95$		
		命中率 (%)	平均费用 (美元)	执行时间 (秒)	命中率 (%)	平均费用 (美元)	执行时间 (秒)	命中率 (%)	平均费用 (美元)	执行时间 (秒)
Θ_8	GreedyCost	0.0	0.21	0.02	0.0	0.21	0.02	0.0	0.21	0.02
	HEFT	99.9	1.19	0.02	99.9	1.19	0.02	99.9	1.19	0.02
	MOHEFT	54.9	0.58	1.78	54.9	0.58	1.82	54.9	0.58	1.80
	Dyna-50k	0.0	1.62	3600.24	0.0	1.62	3600.34	0.0	1.62	3600.65
	Dyna-100k	0.0	1.62	3600.48	0.0	1.62	3600.33	0.0	1.62	3600.43
	Genetic-50k	70.3	2.92	1248.41	86.2	3.51	1245.23	90.8	4.17	1259.34
	Genetic-100k	67.4	1.87	2532.44	93.1	2.59	2468.63	95.2	3.01	2571.68
	EPOSS	77.4	1.13	24.05	96.5	0.84	24.75	96.5	0.84	24.75
	P-EPOSS	76.2	0.96	13.91	97.5	0.70	13.10	97.5	0.70	13.02
Θ_{13}	GreedyCost	0.0	0.21	0.04	0.0	0.21	0.04	0.0	0.21	0.04
	HEFT	99.9	1.19	0.04	99.9	1.19	0.04	99.9	1.19	0.04
	MOHEFT	56.7	1.20	6.23	56.7	1.20	6.20	56.7	1.20	6.24
	Genetic-50k	77.6	2.94	1201.08	87.7	3.54	1189.52	89.3	4.05	1182.60
	Genetic-100k	87.1	2.65	2394.66	92.6	3.52	2467.76	84.6	5.05	2409.78
	EPOSS	80.2	1.26	36.95	90.7	0.77	35.94	95.5	0.62	34.08
	P-EPOSS	89.6	0.83	25.81	95.4	0.63	26.05	95.4	0.63	25.83
Θ_{21}	GreedyCost	0.0	0.21	0.08	0.0	0.21	0.08	0.0	0.21	0.08
	HEFT	99.9	1.19	0.07	99.9	1.19	0.07	99.9	1.19	0.07
	MOHEFT	58.1	1.11	9.93	58.1	1.11	9.99	58.1	1.11	10.03
	Genetic-50k	73.4	3.16	1234.89	86.4	4.06	1207.79	89.6	3.89	1207.80
	Genetic-100k	71.0	2.55	2415.93	87.1	4.09	2373.94	92.6	2.38	2345.93
	EPOSS	80.0	1.26	78.67	95.6	0.61	69.50	95.6	0.61	69.56
	P-EPOSS	86.1	0.71	46.04	99.4	1.03	42.82	99.4	1.03	42.79

表 4: SIPHT 工作流的实验结果。

VM 集合	算法	$p_T = 0.75$			$p_T = 0.9$			$p_T = 0.95$		
		命中率 (%)	平均费用 (美元)	执行时间 (秒)	命中率 (%)	平均费用 (美元)	执行时间 (秒)	命中率 (%)	平均费用 (美元)	执行时间 (秒)
Θ_8	GreedyCost	0.8	0.61	0.02	0.8	0.61	0.02	0.8	0.61	0.02
	HEFT	96.0	8.77	0.02	96.0	8.77	0.02	96.0	8.77	0.02
	MOHEFT	49.8	1.05	2.23	49.8	1.05	2.30	49.8	1.05	2.25
	Dyna-50k	1.9	4.02	3601.09	1.9	4.02	3601.59	1.9	4.02	3601.61
	Dyna-100k	1.9	4.02	3601.53	1.9	4.02	3602.01	1.9	4.02	3601.26
	Genetic-50k	84.0	1.68	3117.24	91.1	2.07	3083.85	94.6	2.83	3046.79
	Genetic-100k	81.3	2.11	6500.74	92.5	3.11	6382.43	95.7	2.80	6308.65
	EPOSS	76.6	0.96	21.29	91.8	1.66	21.37	96.0	9.24	20.62
	P-EPOSS	76.6	0.96	16.57	93.1	1.17	16.12	95.3	1.38	15.98
Θ_{13}	GreedyCost	0.8	0.61	0.04	0.8	0.61	0.04	0.8	0.61	0.04
	HEFT	96.0	8.77	0.04	96.0	8.77	0.04	96.0	8.77	0.04
	MOHEFT	49.8	1.05	7.39	49.8	1.05	7.34	49.8	1.05	7.37
	Genetic-50k	82.1	2.21	3108.30	91.9	1.99	3083.70	94.8	4.01	3102.19
	Genetic-100k	93.0	2.79	6470.66	87.6	1.50	6422.89	94.9	1.93	6289.47
	EPOSS	76.6	0.96	31.23	91.8	1.66	33.25	96.0	9.02	30.85
	P-EPOSS	76.6	0.96	25.37	93.1	1.17	23.41	95.5	1.37	23.33
Θ_{21}	GreedyCost	0.8	0.61	0.07	0.8	0.61	0.07	0.8	0.61	0.07
	HEFT	96.0	8.77	0.07	96.0	8.77	0.07	96.0	8.77	0.07
	MOHEFT	49.8	1.05	14.18	49.8	1.05	14.12	49.8	1.05	14.14
	Genetic-50k	86.6	2.55	3109.83	90.4	2.37	3117.49	94.2	2.50	3093.15
	Genetic-100k	87.0	2.91	6348.05	95.9	3.36	6476.33	95.5	2.78	6309.11
	EPOSS	76.6	0.96	72.72	91.8	1.66	78.42	96.3	11.03	70.41
	P-EPOSS	76.6	0.96	62.90	93.1	1.16	64.96	95.5	1.37	64.36

图 1: CyberShake 工作流的总完工时间、货币费用和调度算法运行时间图示 ($p_T = 0.9$)。

图中术语: Makespan (s) 为总完工时间 (秒); VM Types 为 VM 类型数量; Cost (\$) 为费用 (美元); Alg. Exec. Time (s) 为算法运行时间 (秒); Feasible、Unfeas.、Deadline 分别表示可行、不可行和截止期。

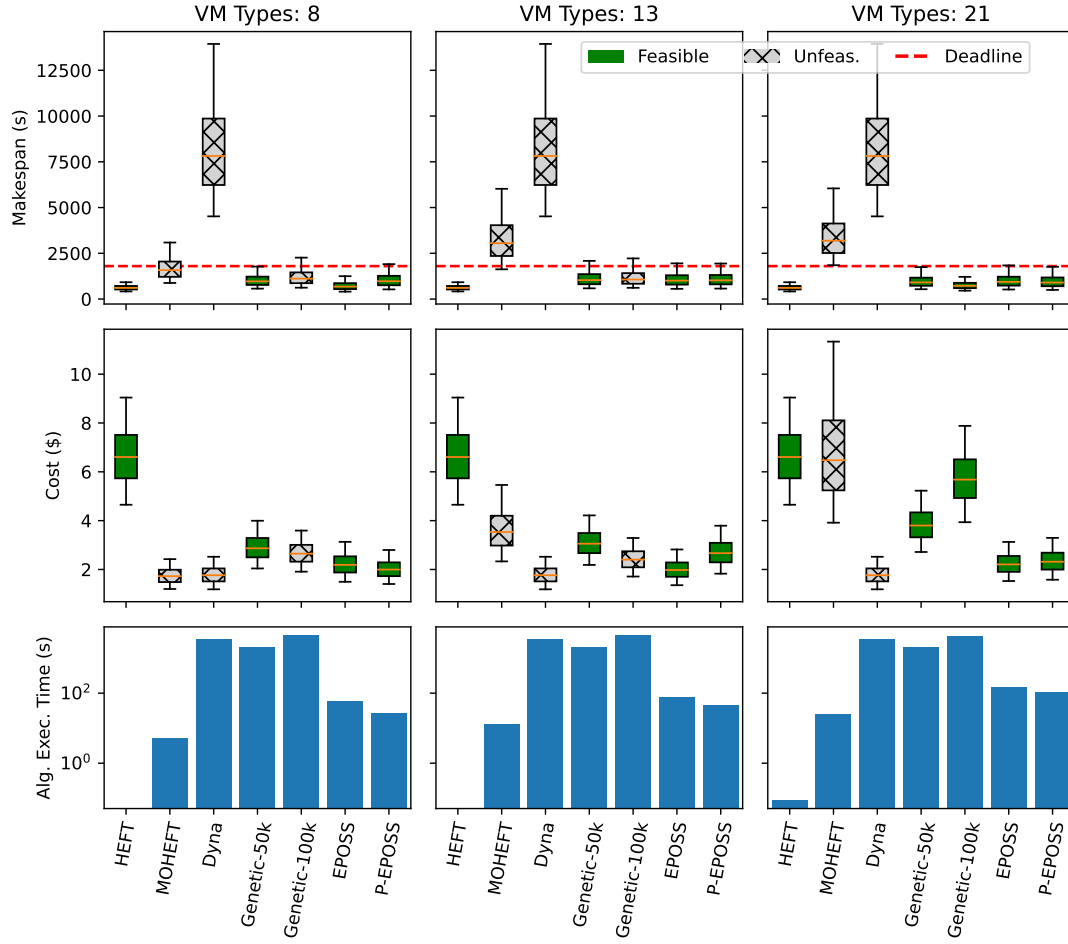


图 2: LIGO 工作流的总完工时间、货币费用和调度算法运行时间图示 ($p_T = 0.9$)。

图中术语: Makespan (s) 为总完工时间 (秒); VM Types 为 VM 类型数量; Cost (\$) 为费用 (美元); Alg. Exec. Time (s) 为算法运行时间 (秒); Feasible、Unfeas.、Deadline 分别表示可行、不可行和截止期。

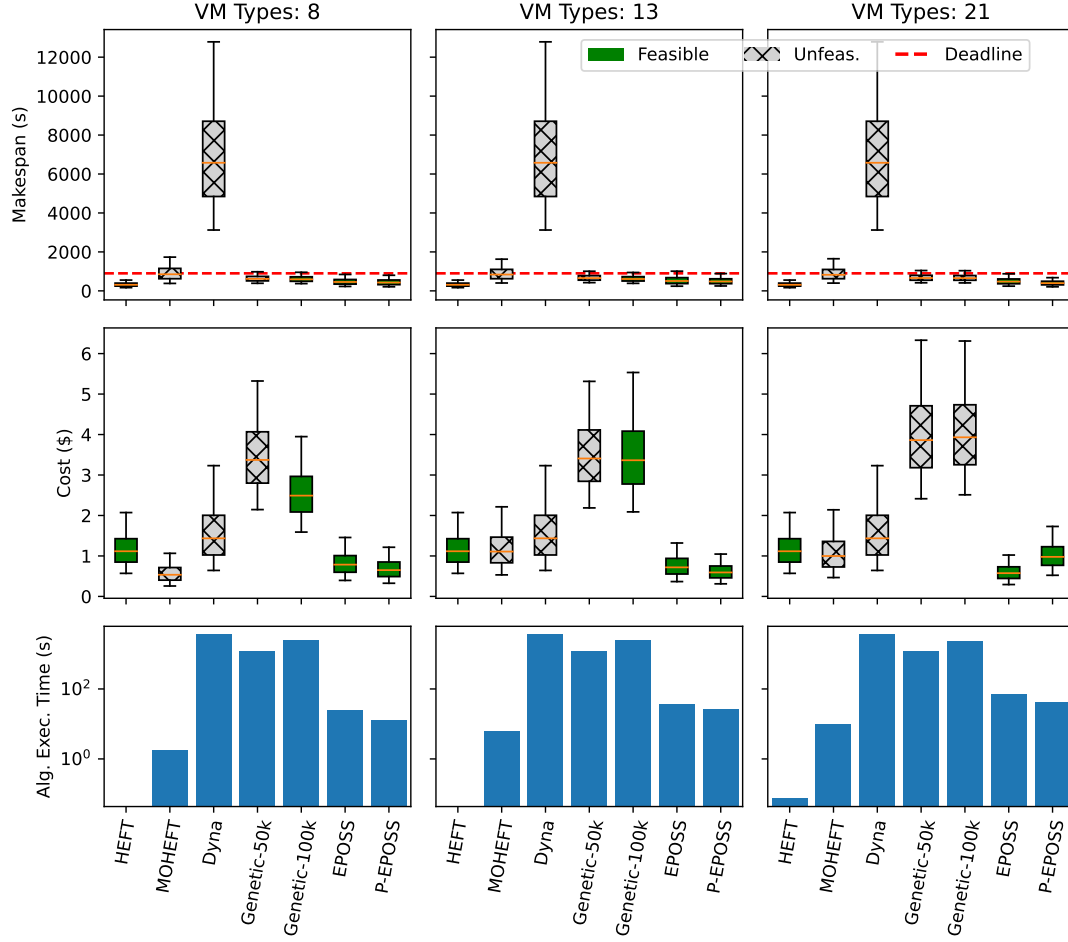


图 3: Montage 工作流的总完工时间、货币费用和调度算法运行时间图示 ($p_T = 0.9$)。

图中术语: Makespan (s) 为总完工时间 (秒); VM Types 为 VM 类型数量; Cost (\$) 为费用 (美元); Alg. Exec. Time (s) 为算法运行时间 (秒); Feasible、Unfeas.、Deadline 分别表示可行、不可行和截止期。

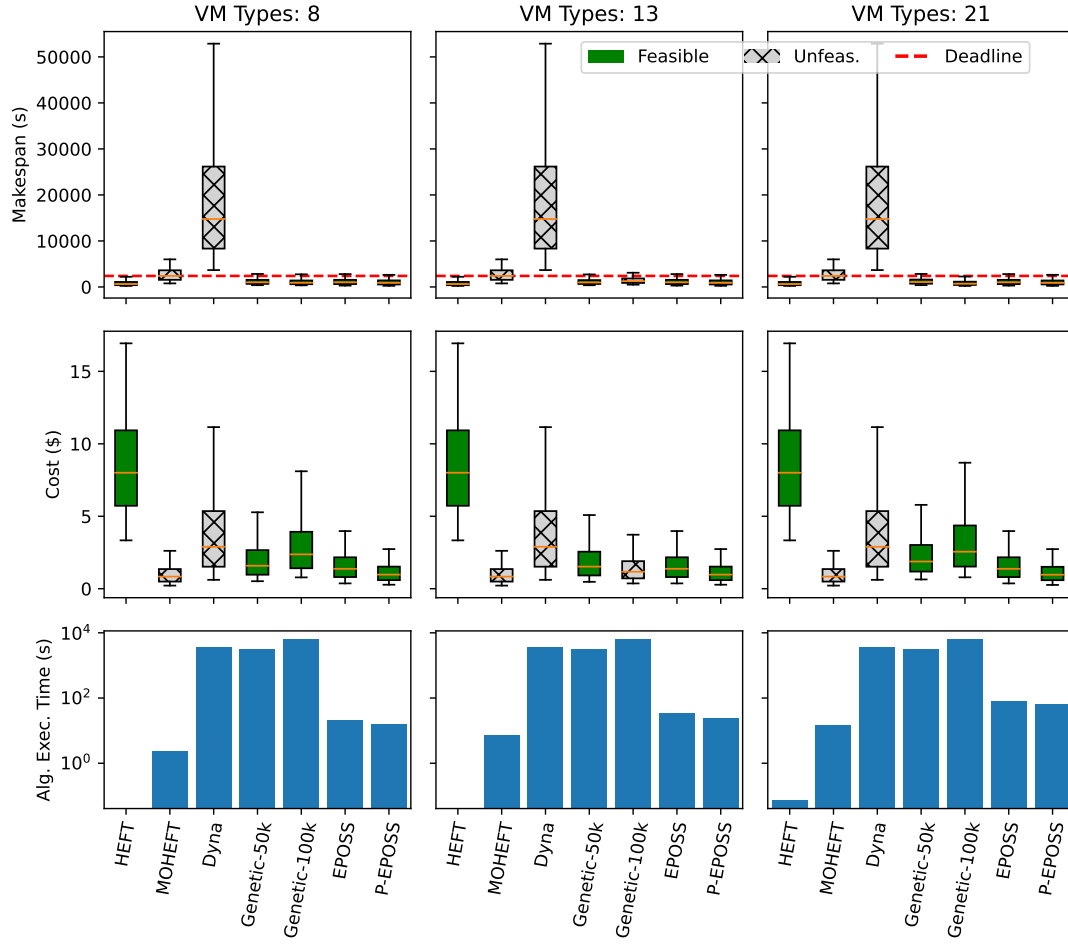


图 4: SIPHT 工作流的总完工时间、货币费用和调度算法运行时间图示 ($p_T = 0.9$)。

图中术语: Makespan (s) 为总完工时间 (秒); VM Types 为 VM 类型数量; Cost (\$) 为费用 (美元); Alg. Exec. Time (s) 为算法运行时间 (秒); Feasible、Unfeas.、Deadline 分别表示可行、不可行和截止期。